Université de Rennes

Master Calcul Scientifique et Modélisation
Computational Science and Modelling
2025–2026

An introduction for scientists to the art of programming [in C major]

Mariko Dunseath Terao

Université de Rennes

Contents

Chapter 1

Introduction

La programmation scientifique est l'art d'utiliser les ressources informatiques - ordinateurs, programmes - pour résoudre des problèmes scientifiques. Ce terme est une traduction imparfaite de l'anglais 'Computational Science', qui inclut aussi la modélisation et les méthodes numériques.

1.1 Objectifs

L'objectif de ce cours est de vous apprendre à développer des codes corrects, à les valider, à les rendre plus efficaces, adaptables et portables, et *in fine* à faire de vous des scientifiques sachant programmer, très demandés par les laboratoires de recherche et entreprises de haute technologie.

Tout comme connaître le vocabulaire et les règles de grammaire d'une langue ne suffit pas pour communiquer, l'apprentissage d'un langage de programmation ne peut se limiter à la syntaxe. Il est malheureusement très facile d'écrire des code faux ou inefficaces. De plus, l'ordinateur étant un outil conçu par d'autres humains et donc perfectible, l'apprentissage intuitif comme en sport ou en musique n'est pas efficace.

Le point fort du cours est de vous faire pratiquer la programmation sur des exercices pertinents, grâce auxquels vous saisirez rapidement les mécanismes sous-jacents. Acquérir les bons automatismes fera de vous un virtuose, maîtrisant suffisamment les contraintes techniques pour pouvoir concentrer votre énergie à la résolution de problèmes scientifiques et au-delà, à la créativité en recherche.

Les cours seront ponctués de nombreux conseils personnels, recettes et astuces du métier. Au travers d'exemples soigneusement sélectionnés, issus de nos expériences professionnelles, vous acquerrez des compétences qui autrement vous demanderaient des années : déchiffrer les codes même très gros, valider les résultats, détecter les sources d'erreur et les corriger rapidement, améliorer leur clarté et performance, ajouter des modules.

Le cours est mis en application *via* le langage de programmation `C`, un choix imposé ici pour des raisons pratiques. La plupart des concepts présentés dans ce cours restent valables pour tous les autres langages.

L'ordinateur n'a aucune intelligence, il n'exécute que ce qu'on lui demande de faire, à une vitesse incroyablement élevée mais sans état d'âme ; c'est donc aux humains de savoir en faire bon usage.

1.2 Mode d'emploi

Les notions sont abondamment illustrées par des exemples en langage `C`. Certains codes mettent en évidence de bonnes pratiques à suivre tandis que d'autres mettent en garde contre des pièges à éviter. Il faut donc bien comprendre les objectifs et signification des codes utilisés, même si les noms des variables et les opérations numériques n'ont pas de motif scientifique profond.

Le C est un langage de programmation

a) impératif : le code décrit des opérations sous forme de séquence d'instructions capables de modifier l'état du programme ;

b) compilé : le code doit être traduit en instructions compréhensibles par l'ordinateur et exécutable indépendamment de tout autre programme à l'exception du système d'exploitation. Le compilateur utilisé dans ce cours-ci est celui du système d'exploitation GNU [1] : `gcc` [2].

Chaque chapitre est accompagné par une archive des codes d'exemple, telle que `Basic.zip`. Elles sont à télécharger de la plate-forme Moodle `Programmation scientifique` de l'Université de Rennes à laquelle vous aurez accès après inscription au cours.

1. Décompresser l'archive avec la commande `unzip Basic.zip` ;
2. Aller dans le dossier `Basic` : `cd Basic` ;
3. Chaque notion présentée dans les notes de cours est mise en situation dans un ou plusieurs codes ;
4. Afficher le code et décortiquer chaque ligne ;
5. Compiler le code avec la commande `gcc` :
   1. Création de l'objet `1_welcome.o`
      `gcc -c 1_welcome.c`
   2. Edition de lien et création de l'exécutable `1_welcome.exe`
      `gcc 1_welcome.o -o welcome.exe`

   Ces deux étapes peuvent être réalisée en une seule avec l'instruction
   `gcc 1_welcome.c -o welcome.exe`
   Si l'on omet `-o welcome.exe`, on obtient un exécutable de nom `a.out`.
6. Exécuter le code avec la commande `welcome.exe`, ou le cas échéant `a.out` ;
7. Analyser le code en fonction des résultats affichés à l'écran par le code ;
8. Procéder de façon similaire pour tous les codes fournis en exemple. Entrer le cas échéant les données demandées à l'écran. Certains comportent des exercices dont l'énoncé est en commentaire. Modifier le code et reprendre le mode d'emploi au point 5.
9. Si le programme est divisé en plusieurs fichiers, par exemple `fich1.c`, `fich2.c`, `fich3.c`, il faut créer les objets pour chaque fichier et ensuite créer l'exécutable :
   `gcc fich1.o fich2.o fich3.o -o prog.exe`
   ou
   `gcc fich1.c fich2.c fich3.c -o prog.exe`

[1] `https://www.gnu.org/`
[2] GNU Compiler Collection `https://gcc.gnu.org/`

Chapter 2

Bases de C

Tout code, aussi complexe soit-il, n'est qu'une combinaison d'instructions élémentaires. Ce chapitre est consacré aux notions de base indispensables pour programmer en `C` :

1. Impression de résultats. Les entrées, complexes en `C`, sont reportées aux chapitres suivants ;
2. Définition de variables ;
2. Opérations arithmétiques ;
3. Condition : le même code fournit des résultats différents car les données varient ;
4. Boucles : un programme exécute de nombreuses instructions si celles-ci sont répétitives.

2.1 Premier programme : Hello world

```c
#include <stdio.h>
int main()
{
  printf ("Welcome to the University of Rennes\n");
  return 0;
}
```

Basic/1_welcome.c

1. Le fichier d'en-tête `stdio.h` (Standard Input/Output Header) inclus par *#include* définit le vocabulaire utilisé par les fonctions d'entrée et de sortie telles que `printf` de la bibliothèque standard du langage. Les fichiers d'en-tête du langage C sont habituellement dans le répertoire `/usr/include`.
2. Le programme doit contenir une seule fonction `main()`
3. Les commandes sont entre accolades { }
4. Une commande se termine par `;`
5. \n dans l'argument de `printf` indique un retour à la ligne
6. Le type de la fonction est `int` car elle renvoie l'entier 0 à la fin de l'exécution.

```c
#include <stdio.h>
int main()
{
  printf ("Welcome ");
  printf ("to the University");
  printf (" of Rennes!");
  return 0;
}
```

Basic/2_welcome2.c

2.2 Commentaires

```c
// What does this code do?
main(){}
```

Basic/3_tiny.c

Les commentaires, ignorés par le compilateur mais utiles pour la lecture du code, sont introduits par //.

2.3 Directive define

1. Format : `#define CONSTANTE VALEUR`
2. Directive du préprocesseur C qui remplace toutes les occurences, à l'exception de celles dans les chaînes de caractères, de CONSTANTE par VALEUR
3. Désavantage : substitution brute du type copié-collé, VALEUR doit être fixé avant la compilation

```c
#include <stdio.h>
#define Pi 3.14  // Pi is a symbolic constant
int main()
{
  double pi2;
  printf("Pi = %lf\n\n",Pi);
  pi2 = Pi * Pi;
  printf("pi2 = %lf\n\n",pi2);
  return 0;
}
// Modify this code so that it prints the results with 6 correct digits
// Test by comparing with M_PI from math.h (add #include <math.h>)
```

Basic/4_define.c

2.4 Variables

Une variable est un objet repéré par un nom, pouvant contenir une ou des données.

1. Le nom doit commencer par une lettre ou _, il peut comporter des lettres, des chiffres et _.
2. Sa longueur est illimitée mais seuls les 32 premiers caractères sont pris en compte.
3. Le langage fait la différence entre minuscule et majuscule : `Val` $\neq$ `val` $\neq$ `VAL`.
4. Certains noms sont interdits (noms réservés) : `auto`, `break`, `case`, `char`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extern`, `float`, `for`, `goto`, `if`, `int`, `long`, `register`, `return`, `short`, `signed`, `sizeof`, `static`, `struct`, `switch`, `typedef`, `union`, `unsigned`, `void`, `volatile`, `while`.
5. Les variables sont stockées en mémoire sur un nombre défini de bits
   a) `short int` : entier court
   b) `int` : entier
   c) `unsigned int` : entier non signé
   d) `long int`, `long long int` : entier (très) long
   e) `unsigned long int`, `unsigned long long int` : entier (très) long non signé
   f) `float` : réel simple précision
   g) `double` : réel double précision
   h) `char` : caractère
6. Avant toute utilisation, une variable **doit** avoir été déclarée, avec un type et un nom. Pour faciliter les opérations numériques, des conversions automatiques de type peuvent être réalisées selon des règles qu'il faut connaître pour éviter les erreurs.
7. Le symbole `=` est utilisé pour **affecter** la valeur à droite du signe à la variable à gauche.
8. On peut initialiser une variable lors de sa déclaration.
9. Une variable doit être imprimée avec le format adéquat, par exemple %d pour `int`, %u pour `unsigned int`, %f pour `float`, %lf pour `double`, %c pour un caractère, %s pour une chaîne de caractères, etc.
10. Le nombre d'octets occupés par une variable est obtenue avec la fonction `sizeof`.
11. L'adresse en mémoire d'une variable est fournie par l'opérateur unaire &. Le format pour l'imprimer est %p, l'adresse est imprimée dans le système hexadécimal.

```c
#include <stdio.h>
int main()
{
  int i=-3;
  float a=1.0/3.0;
  double x=1.0/3.0;
  printf("i=%d size=%ld address=%p \n",i,sizeof(i),&i);
  printf("a=%lf size=%ld address=%p\n",a,sizeof(a),&a);
  printf("x=%21.15e size=%ld address=%p\n",x,sizeof(x),&x);
}
//what happens if x is printed with format %22.16e?
```

Basic/5_variable_address.c

Des constantes mathématiques courantes sont définies dans le header `math.h`, par exemple `M_PI`= π, `M_E`=e, `M_SQRT_2`= $\sqrt{2}$, ...

```c
#include <stdio.h>
#include <math.h>
// Compile with gcc 5_types.c -lm
int main()
{
  short int i=1; int j=1; unsigned int k=1; long int l=1;
  unsigned long int m=1; long long n=1; unsigned long long o=1;
  float a=M_PI;
  double b=M_PI;
  char C[6]="hello";
  printf("                    short int                    int          unsigned int
    long int     long unsigned int             long long    unsigned long long\n");
  int counter;
  for (counter=1;counter < 65;counter++) {
      i *= 2; j *= 2; k *= 2; l *= 2; m *= 2; n *= 2; o *= 2;
      printf(" 2**%2d %20d %20d %20u %20ld %lu %20lld %20llu\n",
      counter,i,j,k,l,m,n,o);}
  printf("a = %23.16f\n",a);
  printf("b = %23.16lf\n",b);
  printf("C = %s\n",C);
  return 0;
}
//How do you explain that some integers appear to be negative?
```

Basic/6_type.c

2.5 Opérations

Les opérateurs arithmétiques courants sont

`+`	addition	`*`	multiplication	`%`	modulo
`-`	soustraction	`/`	division	`=`	affectation

1. Une opération entre entiers donne un entier, par exemple `3/2 = 1`, `4/5 = 0`
2. Le type d'opération dépend du type des opérands, pas du type de variable où est stocké le résultat
   ```
   double x;
   int i=1,j=5;
   x = i/j;
   ```
   La valeur de x sera 0 et non 0.2
3. Conversion en double : si un des opérands est double, le résultat sera double
   ```
   double x;
   x = 1.0/5;
   x = 1/5.0;
   x = 1.0 / 5.0;
   ```
   La valeur de x sera 0.2
4. A retenir : il vaut mieux ne pas mélanger les types !
5. Si `x` est double, `x = 1.0` est préférable à `x = 1`

```c
#include <stdio.h>
int main()
{
  int m = 11, n = 5;
  printf("m = %d, n = %d\n", m, n);
  printf("m + n = %d\n", m + n);
  printf("m - n = %d\n", m - n);
  printf("m * n = %d\n", m * n);
  printf("m / n = %d\n", m / n);
  printf("m %% n = %d\n\n", m % n);
  m = -11;
  printf("m = %d, n = %d\n", m, n);
  printf("m + n = %d\n", m + n);
  printf("m - n = %d\n", m - n);
  printf("m * n = %d\n", m * n);
  printf("m / n = %d\n", m / n);
  printf("m %% n = %d\n\n", m % n);  /* printing out % symbol */
  m = 1;
  printf("m = %d, n = %d\n", m, n);
  printf("%d / %d = %d\n\n", m, n, m / n);
  double x;
  x = m / n;
  printf("x = %d / %d = %lf\n", m, n, x);
  printf ("\n1.0/5 = %lf\n1/5.0 = %lf\n1.0/5.0 = %lf\n\n",1.0/5,1/5.0,1.0/5.0);
  return 0;
}
```

Basic/7_int_operator.c

2.6 Conversion

1. Un double converti en float perd de la précision
2. Les décimales, une fois perdues, ne sont pas récupérables

```c
#include <stdio.h>
#include <math.h>
// Compile with gcc 8_convert.c -lm
int main()
{
  float a;
  double x;
  x = acos(-1.0);
  printf ("    x = %23.16e\n",x);
  printf (" M_PI = %23.16e\n",M_PI);
  a = x;
  printf ("    a = %23.16e\n",a);
  x = a;
  printf ("    x = %23.16e\n",x);
  return 0;
}
```

Basic/8_convert.c

2.7 Arrondi

1. Un arrondi vers un entier positif se fait vers le plus petit :
   `(int) 0.25 = (int) 0.75 = 0`
2. Un arrondi vers un entier négatif se fait vers le plus grand :
   `(int) -1.25 = (int)-1.75 = -1`

```c
#include <stdio.h>
int main()
{
  double x,y,z;
  x = 1000.0/3.0;
  y = x - 333.0;
  z = 3.0 * y - 1.0;
  printf(" x = %23.16e\n",x);
  printf(" y = %23.16e\n",y);
  printf(" z = %23.16e\n",z);
  x = 0.25;
  printf("(int)%lf = %d\n",x,(int)x);
  x = 0.75;
  printf("(int)%lf = %d\n",x,(int)x);
  y = -1.25;
  printf("(int)%lf = %d\n",y,(int)-1.25);
  y = -1.75;
  printf("(int)%lf = %d\n",y,(int)y);
  return 0;
}
```

Basic/9_rounding.c

2.8 Limites de la représentation des nombres

1. La représentation des nombres sur un nombre fini de bits impose des limites sur la gamme des nombres représentables et sur leur précision
2. Ces paramètres sont définis dans les headers limits.h et float.h

```c
#include <stdio.h>
#include <limits.h>
// Header file limits.h contains symbolic constants such as INT_MIN, etc
int main()
{
  printf("SHRT_MIN = %d\n", SHRT_MIN);
  printf("SHRT_MAX = %d\n", SHRT_MAX);
  printf("INT_MIN = %d\n", INT_MIN);
  printf("INT_MAX = %d\n", INT_MAX);
  printf("LONG_MIN = %ld\n", LONG_MIN);
  printf("LONG_MAX = %ld\n", LONG_MAX);
  printf("USHRT_MAX = %u\n", USHRT_MAX);
  printf("UINT_MAX = %u\n", UINT_MAX);
  printf("ULONG_MAX = %lu\n", ULONG_MAX);
  return 0;
}
```

Basic/10_int_limit.c

```c
#include <stdio.h>
#include <math.h>
#include <float.h>

// Compile with: gcc 11_real_limit.c -lm

int main()
{
  int n = 12;
  float a, x = 3.1415926;
  double b, pi, y = 3.1415926;

  a = x + n;
  printf(" %f + %d = %f\n",x, n, a);
  printf(" %9.7f + %d = %f\n",x, n, a);

  b = y + n;
  printf(" %9.7f + %d = %f\n",y, n, b);

  pi = acos(-1.);
  printf(" pi    = %23.16e\n",pi);
  printf(" M_PI = %23.16e\n\n",M_PI);

  printf(" FLT_MIN = %15.8e\n",FLT_MIN);
  printf(" FLT_MAX = %15.8e\n",FLT_MAX);
  printf(" FLT_DIG = %d\n",FLT_DIG);
  printf(" FLT_EPSILON = %15.8e\n",FLT_EPSILON);
  printf(" DBL_MIN = %23.16e\n",DBL_MIN);
  printf(" DBL_MAX = %23.16e\n",DBL_MAX);
  printf(" DBL_DIG = %d\n",DBL_DIG);
  printf(" DBL_EPSILON = %23.16e\n",DBL_EPSILON);

  printf (" DBL_EPSILON   + 1.0 = %23.16e + %23.16e = %23.16e\n\n",
    DBL_EPSILON, 1.0, DBL_EPSILON + 1.0);
  printf (" DBL_EPSILON/2 + 1.0 = %23.16e + %23.16e = %23.16e\n\n",
    DBL_EPSILON*0.5, 1.0, DBL_EPSILON*0.5 + 1.0);

  float t;
  t = 256.0*(FLT_MIN/256.0);
  printf ("t = %23.16e\n",t);

  double d;
  d = 256.0*(DBL_MIN/256.0);
  printf ("d = %23.16e\n",d);

  return 0;
}
```

Basic/11_real_limit.c

2.9 Opérateur d'affectation =

Exemples :

```
int i;
i = 1;
i = i + 1;
```

1. On évalue d'abord `i + 1` $= 1 + 1 = 2$, ensuite on affecte cette valeur à la variable `i`. L'ancienne valeur de `i` est perdue.
   = n'est pas l'opérateur égalité !
2. On peut affecter la même valeur à plusieurs variables
   ```
   i = j = k = 5;
   ```
3. `x =+y` affecte la valeur de `y` à `x`
   `x =-y` affecte la valeur de `-y` à `x`
4. `x O=n` est équivalent à `x = x O n` où `O` peut être l'opérateur `+,-,*,/`.

```c
#include <stdio.h>
int main()
{
  int i,j,k;

  i = 1;
  printf("i = %d\n", i);
  i = i + 1;
  printf("i=i+1 : i=%d\n\n", i);

  i = j = k = 5;
  printf("i = j = k = 5 :\n");
  printf("i = %d\nj = %d\nk = %d\n\n",i,j,k);

  i += 3;
  printf("i += 3 : i = %d\n", i);
  i -= 1;
  printf("i -=1 : i = %d\n", i);
  i *= 7;
  printf("i *= 7 : i = %d\n", i);
  i /= 5;
  printf("i /=5: i = %d\n", i);

  i =+ 3;
  printf("i =+3 : i = %d\n", i);

  i =- 3;
  printf("i = -3 : i = %d\n", i);

  return 0;
}
```

Basic/12_affect.c

2.10 Opérateurs d'incrémentation

1) ++ augmente d'une unité la variable
2) −− diminue d'une unité la variable
3) Post-in/decrement : m++, m−−
4) Pre-in/decrement : ++m, −−m

```c
#include <stdio.h>
int main()
{
  int m = 5;
  int n1, n2;

  printf("m = %d\n", m);
  m++;
  printf("m++ : m=%d\n", m);
  m--;m--;
  printf("m--;m-- : m=%d\n\n", m);

  printf("Post- increment and decrement\n");
  n1 = m++;
  printf("n1=m++: n1 = %d  m = %d\n", n1, m);
  n2 = m--;
  printf("n2=m--: n2 = %d  m = %d\n\n", n2, m);

  printf("Pre- increment and decrement\n");
  n1 = ++m;
  printf("n1=++m: n1 = %d  m = %d\n", n1, m);
  n2 = --m;
  printf("n2=--m: n2 = %d  m = %d\n\n", n2, m);

  return 0;
}
```

Basic/13_inc.c

2.11 Ordre des opérations arithmétiques

Priorité des opérations

1. Multiplication et division sont prioritaires par rapport à addition et soustraction
2. Les opérations sont exécutées de gauche à droite

a + b*c est égal à a + (b*c)
a + b/c est égal à a + (b/c)
a/b/c est égal à (a/b)/c

On peut modifier l'ordre d'exécution d'opérations à l'aide de parenthèses, par exemple :
(a + b) * c
(a + b) / c
a/(b/c)

```c
#include <stdio.h>
int main()
{
  int n1, n2, n3, n4, n5;

  n1 = 3 * 4 + 2;
  printf("3 * 4 + 2 = %d\n", n1);
  n2 = -3 * 4 + 2;
  printf("-3 * 4 + 2 = %d\n", n2);
  n3 = 15 / 3 + 2;
  printf("15 / 3 + 2 = %d\n", n3);
  n1 = 20; n2=5; n3=2;
  n4 = n1 / n2 / n3;
  printf("%d / %d / %d = %d\n", n1,n2,n3,n4);
  n4 = n1 / (n2 / n3);
  printf("%d / (%d / %d) = %d\n\n", n1,n2,n3,n4);

  printf("n1 = %d n2 = %d n3 = %d\n",n1,n2,n3);
  n4 = ++n3 * --n2;
  printf(" ++n3 * --n2 = %d\n", n4);
  n5 = n3 + 1 * n2 - 1;
  printf("n3 + 1 * n2 - 1 = %d\n", n5);
  printf (" ++n1 = %d  ++n1 = %d  ++n1 = %d\n\n",++n1,++n1,++n1);

  n1 = 3;
  printf("n1 = %d\n",n1);
  n2 = ++n1 + ++n1;
  printf("++n1 + ++n1 = %d\n",n2);
  printf("n1 = %d\n",n1);

  n3 = ++n1 - ++n1;
  printf("++n1 - ++n1 = %d\n",n3);
  printf("n1 = %d\n",n1);

  n4 = ++n1 * ++n1;
  printf("++n1 * ++n1 = %d\n",n4);
  printf("n1 = %d\n",n1);

  n5 = ++n1 / ++n1;
  printf("++n1 / ++n1 = %d\n",n5);
  printf("n1 = %d\n",n1);

  return 0;
}
```

Basic/14_precedence.c

2.12 Fonctions mathématiques simples

Pour pouvoir utiliser les fonctions mathématiques élémentaires, il faut ajouter le header `math.h` et compiler avec l'option `-lm` pour éditer le lien avec la bibliothèque mathématique.

```c
#include <stdio.h>
#include <math.h>

// Compile with: gcc mathfunc.c -lm

int main()
{
  double a, b, pi, x;

  pi = acos(-1.);
  printf(" pi    = %23.16e\n",pi);

  a = cos(pi);
  printf(" cos(%e) = %23.16e\n",pi,a);

  a = sin(pi/3.0);
  b = cos(pi/3.0);
  printf(" a = sin(pi/3.0) = %23.16e\n b = cos(pi/3.0) = %23.16e\n",a,b);
  printf(" a**2 +b**2 = %23.16e\n",pow(a,2)+pow(b,2));

  return 0;

  /* Main mathematical functions
      fabs(x)
      pow(x,y), sqrt(x)
      exp(x),   log(x),  log10(x)
      sin(x),   cos(x),  tan(x)
      asin(x),  acos(x), atan(x), atan2(y,x)
      sinh(x),  cosh(x), tanh(x)
      ceil(x),  floor(x)
  */
}
```

Basic/15_mathfun.c

2.13 Conditions et opérateurs de comparaison

```c
#include <stdio.h>
#include <string.h>
int main()
{
  int i,j;
  printf(" Please enter two integers\n");
  scanf("%d%d",&i,&j);
  printf(" The maximum is ");
  if (i > j) printf ("%d\n",i);
  else printf ("%d\n",j);
  printf(" Enter one of the following integers: (0,1,2)\n");
  scanf("%d",&i);

  char letter[1];
  printf("Please Enter one of the following letters (A,B,C)\n");
  scanf("%s",letter);
  if (letter[0] == 'A')
    {
      printf(" First case\n");
      printf(" You write A\n");
    }
  else if (letter[0] == 'B')
    {
      printf(" Second case\n");
      printf(" You wrote B\n");
    }
  else if (letter[0] == 'C')
    {
      printf(" Third case\n");
      printf(" You wrote C\n");
   }
  else
    {
      printf("You must enter one of the following letters (A,B,C)!!\n");
    }
  return 0;
}
/*
 Operators of comparison :
 equal            : ==
 different        : !=
 superior         : >
 superior or equal: >=
 inferior         : <
 inferior or equal: <=
 Combinaison of comparisons :
 logical and      : &&
 logical or       : ||
*/
```

Basic/16_condition.c

NB: A cause des erreurs d'arrondi, il ne faut jamais tester l'égalité de deux réels : if (theta == M_PI) ... il faut tester : if (fabs(theta-M_PI) < 1e-15) ...

1. 0 est faux
2. toute valeur non nulle est vraie

```c
#include <stdio.h>
int main()
{
  int i=0,j=1;
  printf(" i = %d, j = %d\n",i,j);
  if (i=0) printf("Is i=0? i = %d\n",i);
  else printf (" i = %d\n",i);
  printf("We still have i = %d, j = %d\n",i,j);
  if (i=j) printf(" i = %d, j = %d\n",i,j);
  return 0;
}
```

Basic/17_horrors.c

2.14 Switch

```c
#include <stdio.h>
int main(){
  int i;
  printf(" Enter one of the following integers: (0,1,2)\n"); scanf("%d",&i);
  switch(i) {
    case 0: printf(" a\n"); break; // if i=0, the following instructions are not executed
    case 1: printf(" b\n");        // the following instructions are executed
    case 2: printf(" c\n");        // the following instruction is executed
    default: printf(" Vous devez entrer un entier parmi (0,1,2)\n\n");
    }
  return 0;
}
```

Basic/18_switch.c

2.15 Opérateur ternaire d'évaluation conditionnelle

`a?b:c` si a est faux (c-à-d 0), le résultat est c, sinon le résultat est b

```c
#include <stdio.h>
int main(void){
  float min,x=0.0,y=1.0,z=2.0;
  printf(" x = %lf\n",x); printf(" y = %lf\n",y); printf(" z = %lf\n",z);
  printf (" x ? y:z %f\n",x?y:z);
  printf (" z ? x:y %f\n",z?x:y);
  printf("enter real x: "); scanf("%f",&x);
  printf(" x = %lf\n",x); printf("enter real y: ");
  scanf("%f",&y); printf(" y = %lf\n",y);
  min = ((x<y) ? x:y); printf(" minimum of (%f,%f) is %f\n",x,y,min);
  return 0;
}
```

Basic/19_conditional.c

2.16 Répétition d'instructions par incrément

Syntaxe: `for(initialisations;conditions de répétition;incrémentations) instruction;`

Exemples :

```
for(i=0;i<i_max; i++)
   {
    liste d'instructions ;
   }
for(i=0,j=20;i<i_max; i++,j--,a=b)
   {
    liste d'instructions ;
   }
```

```c
#include <stdio.h>
int main()
{
  int i, N, s, p;
  N = 10;
  for (i = 1, s = 0, p = 1; i <= N; i++)
    {
      s += i;
      p *= i;
    }
  printf(" Sum %d   Product %d\n",s,p);
  printf("After the loop, i = %d\n",i);
  return 0;
}
```

Basic/20_loop.c

2.17 Répétition d'instructions par test

Lorsque la condition d'arrêt de la boucle est calculée, c-à-d n'est pas connue *a priori* :

```c
#include <stdio.h>
int main()
{
  int i=1, N, s=0, p=1;
  printf("Enter maximum value of sum: ");
  scanf("%d",&N);
  while (s+i <= N)
    {
      s += i;
      p *= i;
      i++;
    }
  printf("%d ! = %d, sum of factors = %d\n",i-1,p,s);
  return 0;
}
```

Basic/21_while.c

Lorsque les instructions de la boucle sont à exécuter au moins une fois :

```c
#include <stdio.h>
int main()
{
  int i=1, N, s=0, p=1;
  printf("Enter maximum value of sum: ");
  scanf("%d",&N);
  do
    {
      s += i;
      p *= i;
      i++;
    }
  while (s+i <= N);
  printf("%d ! = %d, sum of factors = %d\n",i-1,p,s);
  return 0;
}
```

Basic/22_do_while.c

2.18 Interruptions de boucle

```c
#include <stdio.h>
int main()
{
  int i;
  printf ("for loop:\n");
  for (i=1;i<10;i++) {
      printf("i = %d\n",i);
      if (i<3) continue; if (i>7) break;
      printf ("hello\n");
    }
  printf ("\nwhile loop:\n");
  i = 0;
  while (i<10) {
      i++; printf("i = %d\n",i);
      if (i<3) continue; if (i>7) break;
      printf ("hello\n");
    }
  printf ("\ndo while loop:\n");
  i = 0;
  do {
    i++; printf("i = %d\n",i);
    if (i<3) continue; if (i>7) break;
    printf ("hello\n");
  } while (i<10);
  return 0;
}
```

Basic/23_loop_interrupt.c

`continue` : les instructions suivantes de la boucle ne sont pas exécutées
`break` : la boucle est interrompue, les instructions suivantes de la boucle ne sont pas exécutées.

2.19 Exercices

1. Ecrire un code qui permute deux nombres.
2. Ecrire un code qui détermine si un entier est pair ou impair. Généraliser le code pour déterminer si un entier est un multiple d'un autre entier non nul.
3. Ecrire un code qui imprime les chiffres d'un entier dans l'ordre inverse.
4. Ecrire un code qui calcule le plus grand diviseur commun et le plus petit multiple commun de deux entiers.
5. Ecrire un code qui génère une liste de nombres aléatoires.
6. Nombres de Fibonacci
   (a) Ecrivez un code affichant les M premiers nombres de Fibonacci F_n

   $$0, 1, 1, 2, 3, 5, 8, 13, \ldots$$

   (b) Vérifiez jusqu'à quelle valeur de M votre code est valide.
   (c) Montrez que

   $$\lim_{n\to\infty} \frac{F_n}{F_{n-1}} = \frac{1+\sqrt{5}}{2} \equiv \phi$$

Chapter 3

Data transfer

3.1 Introduction

While developing large codes, people often focus on the algorithms as these are naturally considered the most interesting and challenging part of the work. They thus tend to neglect all the rest, which is a common but very bad practise. It is indeed much more likely that mistakes arise at interfaces, as when a task is distributed between several workers who do not understand the instructions in exactly the same and coherent way. Developing good codes requires in fact

1. a good overall architecture, with well-defined sets of functions performing a common task, keeping also in mind future development. This requires a deep understanding of the problems to be solved, the targets to be achieved, the ressources available (memory, development time and even hardware), the users, etc ...
2. careful I/O: "rubbish in, rubbish out!"; results must be easily exploitable;
3. efficient data transfers between different parts of the code, critical for bug free, adaptable and portable codes; many issues in HPC are linked to memory management.

Many mistakes arise from misunderstanding of inputs and outputs between functions. As long as there is no type-mismatch, they are not detected by the compiler – whose responsability is not to debug anyway. They are hence difficult to fix.

Optimising the transfer of data is important to minimize

1. runtime,
2. memory use,
3. programming errors,
4. development time.

In this chapter, we present different methods of sharing data within a program.

3.2 Scope of a variable: local and global variables

1. A `global` variable is declared outside functions and is accessible from anywhere within the file.
2. A `local` variable is visible in the section between braces where it is defined.
3. It is thus possible to have different variables with the same name! This is obviously a dangerous practise and should be avoided as much as possible.

```c
#include <stdio.h>
int i=12,j;
int main (){
  { int i,j; // i and j are local to the region limited by the closest curly braces
    for (j=1,i=1;i<=5;i++) j = 2*j;
    printf(" i = %2d j = %2d \n",i,j);
  }
  printf(" i = %2d j = %2d \n",i,j); // i and j are the global variables defined before main
  return 0;
}
```

DataTransfer/1_test_global_local.c

```c
#include <stdio.h>
int i = 12;
void test (int i) {printf (" From test i = %d\n",i);}
int main(){
  printf (" First printf i = %d\n",i);
  int i=0;
  printf (" Second printf i = %d\n",i);
  { int i=100; printf (" Third printf i = %d\n",i);}
  printf (" Fourth printf i = %d\n",i);
  test(i);
  return 0;
}
```

DataTransfer/2_scope.c

3.3 Type `static`

A `static` variable conserves its value between two calls of the function

```c
#include <stdio.h>
void fct(void) {
  static int a=0;  // this variable's lifetime extends across the entire run of the program
  a++;             // in contrast to that of local, automatic variables
  printf("From function fct: a= %d\n",a);
}
int main(){
  int i,N;
  printf ("Enter number N of calls of the function incrementing the static variable a: ");
  scanf("%d",&N);
  for (i=0; i<N; i++) fct();
  return 0;
}
```

DataTransfer/3_static.c

A variable can be both `global` and `static`

```c
#include <stdio.h>
static int N;  // Global static variable, visible to all functions within this file
void fct(void) {printf("From function fct: N= %d\n",N); N++;}
int main(){
  int i;
  printf ("Enter the initial value of the global static variable N: ");
  scanf("%d",&N);
  for (i=0;i<5;i++) fct();
  return 0;
}
```

DataTransfer/4_global_static.c

3.4 Headers

All variables must be defined before their first use and this rule applied to functions too.
Native C functions are defined in files called `headers` which can be included in codes using the `#include` directive, for instance

basic mathematical functions	`math.h`
I/O functions	`stdio.h`
functions for characters	`ctype.h`
functions for strings	`string.h`

It is a good practise to define `headers` including the prototypes of the functions used in any program. The following example illustrates what might happen otherwise.

```c
#include <stdio.h>
#include "6_myheader.h"
// To compile: gcc 7_myfunctions.c 8_myfunctions_noh.c 5_test_header.c
int main()
{
  int i,j;
  double a,b;
  i=func_int();
  a=func_double();
  printf("i=%d\n",i);
  printf("a=%lf\n",a);
  printf("What happens without header?!\n");
  j=func_int_noh();
  b=func_double_noh();
  printf("j=%d\n",j);
  printf("b=%lf\n",b);
  return 0;
}
```

DataTransfer/5_test_header.c

```
int func_int();
double func_double();
```

DataTransfer/6_myheader.h

```
#include <math.h>
int func_int(){return 3;}
double func_double(){return M_PI;}
```

DataTransfer/7_myfunctions.c

```
#include <math.h>
int func_int_noh(){return 4;}
double func_double_noh(){return M_PI;}
```

DataTransfer/8_myfunctions_noh.c

NB: A `header` file must never be compiled since it is included in other files that are compiled. The program compiles *albeit* with warnings

```
5_test_header.c: In function ‘main’:
5_test_header.c:13:5: warning: implicit declaration of function ‘func_int_noh’;
did you mean ‘func_int’? [-Wimplicit-function-declaration]
   13 |   j=func_int_noh();
      |     ^~~~~~~~~~~~
      |     func_int
5_test_header.c:14:5: warning: implicit declaration of function ‘func_double_noh’;
did you mean ‘func_double’? [-Wimplicit-function-declaration]
   14 |   b=func_double_noh();
      |     ^~~~~~~~~~~~~~~
      |     func_double
```

and the execution gives

```
i=3
a=3.141593
What happens without header?!
j=4
b=1413754136.000000
```

The value of `b` is wrong.

This example demonstrates that warnings must be treated as seriously as errors.
A code that compiles and runs without error messages is not necessarily correct.
It is up to the code developer/user to verify.

3.5 Type `extern`

When using a variable defined in a different part of the code (typically in a separate file), it must be defined as a variable of type `extern`.

```c
#include <stdio.h>
#include <math.h>
#include "11_myheader.h"
// To compile: gcc 10_externX.c 9_extern.c -lm
int main()
{
  int i,N;
  extern double X;
  printf("This program demonstrates the use of external variables\n");
  printf ("Enter the number of calls N to the function incrementing the variable X: ");
  scanf("%d",&N);
  for (i=0;i<N;i++) {fct(i);printf("From main: cos (%d*M_PI/2) = %lf\n",i,cos(X));}
  return 0;
}
```

DataTransfer/9_extern.c

```c
#include <math.h>
double X; // Global variable visible from all functions using extern double X
void fct(int i) {X=i*0.5*M_PI;}
```

DataTransfer/10_externX.c

```c
void fct(int);
```

DataTransfer/11_myheader.h

3.6 Type `const`

The value of a variable can be protected (*i.e.* is non modifiable) if it is of type `const`

1. Format : `const type variable`
2. If a `constant` variable is modified, an error is detected at compile-time.

```c
#include <stdio.h>
int main()
{
  const int n = 42;
  printf("n = %d\n", n);
  // What is wrong with the following line? Try correcting the mistake
  n = n + 1;
  const double pi = 3.1415;
  printf("pi = %18.16lf\n", pi);
  return 0;
}
```

DataTransfer/12_const.c

3.7 Input and Output

Standard inputs and outputs can simply be redirected to files at execution time, using the < and >

```
a.out < input > output
```

where `input` and `output` are the name of the input and output files.

To redirect I/O inside the code, a pointer to a file must be used. The type of such a pointer is `FILE`. It is made to point to a file with the function `fopen`.

```c
#include <stdio.h>
#include <math.h>
// ASCII input and output in and from a file
int main()
{
  int i = 1;
  double pi =M_PI;
  FILE *fp;
  char filename[10]="io.dat";
  char txt[10];
  fp = fopen (filename,"w");
  if (fp == NULL) {
      fprintf (stderr,"Impossible to open the file %s\n",filename);
      return(1);
  }
  fprintf (fp,"Hello!\n");
  fprintf (fp,"%d %18.15lf\n",i,pi);
  fclose (fp);
  fp = fopen (filename,"a+");
  if (fp == NULL) {
    fprintf (stderr,"Impossible to open the file %s\n",filename);
      return(1);
  }
  fscanf(fp,"%s",txt);
  fscanf(fp,"%d",&i);
  fscanf(fp,"%lf",&pi);
  printf("%s\n",txt);
  printf("i = %d pi = %18.15lf\n",i,pi);
  fprintf(fp,"cos(%d*M_PI)= %18.15lf\n",i,cos(i*pi));
  fclose(fp);
  return 0;
}
/*
Ascii I/O mode: "r" for reading
                "w" for writing
                "a" for writing at the end of the file
                "r+" or "w+" for reading and writing
                "a+" for reading and writing at the end of the file
*/
```

DataTransfer/13_io.c

For large sets of data and when accuracy is required, data must be recorded in binary files.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
// Binary input and output to and from a file
int main()
{
  int i,M,N;
  int *array, *V;
  double pi = M_PI;
  FILE *fp;
  char filename[20] = "binary.dat";
  printf("Enter dimension of array: ");
  scanf("%d",&M);
  array = malloc(M*sizeof(int));
  for (i=0;i<M;i++) array[i]=2.0*i;
  fp = fopen (filename,"wb");
  if (fp == NULL)
    {
      fprintf (stderr,"Impossible to open the file %s\n",filename);
      return(1);
    }
  fwrite (filename,sizeof(char),1,fp);
  fwrite (&M,sizeof(int),1,fp);
  fwrite (&pi,sizeof(double),1,fp);
  fwrite (array,sizeof(int),M,fp);
  fclose (fp);
  fp = fopen (filename,"rb");
  if (fp == NULL)
    {
      fprintf (stderr,"Impossible to open the file %s\n",filename);
      return(1);
    }
  fread (filename,sizeof(char),1,fp);
  fread (&N,sizeof(int),1,fp);
  V = malloc(N*sizeof(int));
  fread (&pi,sizeof(double),1,fp);
  fread (V,sizeof(int),N,fp);
  printf ("N = %d  pi = %18.15lf\n",N,pi);
  printf ("filename = %s\n",filename);
  for (i=0;i<N;i++) printf("V[%d]=%d\n",i,V[i]);
  fclose (fp);
  return 0;
}
/*
Binary I/O modes:
       "rb" for reading
       "wb "for writing
       "ab" for writing at the end of the file
       "r+b", "rb+" or "w+b", "wb+" for reading and writing
       "a+b", "ab+" for reading and writing at the end of the file
*/
```

DataTransfer/14_io_binary.c

3.8 Parameters of the `main` program

It is possible to pass data to the main program by using the following declaration

`int main(int argc, char **argv)` or equivalently `int main(int argc, char *argv[])`

The types `int` and `char **` are compulsary, the name of the variables `argc` and `argv` are conventional and can be chosen by the developer. The parameter `argc` is the number of arguments of the program, which are strings contained in the array `argv[]`. This is an array of strings which are themselves arrays of characters, hence the type `char **/char*[]`.

The first argument `argv[0]` is the name of the executable. The other arguments can be entered at run-time immediately after the name of the executable. These however are all interpreted as strings! To enter numerical parameters, it is necessary to convert the arguments into the desired type using the appropriate function:

converting ASCII to	function
`int`	`atoi`
`double`	`atof`

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 10
// Parameters of a program with cast of char
// Compile with gcc 9_prog_param.c and run for example with :
// a.out 1.0 -1.0 3.141592653589793
int main(int counter, char **parameter)
// int main(int counter, char *parameter[])
{
  int i;
  printf ("This programme has %d arguments :\n",counter);
  for (i=0;i<counter;i++) printf ("parameter[%d]=%s\n",i,parameter[i]);
  printf ("Cast arguments to integers:\n");
  for (i=1;i<counter;i++) printf ("parameter[%d]=%d\n",i,atoi(parameter[i]));
  printf ("Cast arguments to double:\n");
  for (i=1;i<counter;i++) printf ("parameter[%d]=%23.15lf\n",i,atof(parameter[i]));
  printf("Sum of last %d arguments:\n",counter-1);
  double sum=0.0;
  for (i=1;i<counter;i++) sum += atof(parameter[i]); // what happens without conversion?
  printf("sum = %23.15lf\n",sum);
  return 0;
}
```

DataTransfer/15_prog_param.c

This method of passing data to a `main` is particularly useful when calling the code from within another one, such as a graphical interface. Upon successfull execution, the `main` will also return a 0 which will be useful for the calling code, for instance in order to continue execution or to take another action if there is a failure.

Chapter 4

Pointers

4.1 Introduction

In a computer, all information is recorded in its memory. More precisely, any data is encoded as a series of 0 and 1 corresponding to the state OFF or ON of a transistor. A program thus needs to know where the data – including the instructions it is composed of – are stored. At the dawn of the computer age, codes were very laborious to develop and highly machine dependent. The invention of compilers and compiled languages[1] relieved code developers from memory management, leading to software portability.

As the C language was initially designed to implement an operating system[2], it includes native facilities to access and manipulate data addresses. For instance, the address of a variable is provided by simply applying to it the unary operator &. Any non trivial code in C needs **pointers** corresponding to a type of variable that can contain the address of another variable.

In this chapter, we will first (re)discover that in C, parameters to a function are passed by copy of value. This has as consequence the very important fact that to modify a variable through a function, the function needs to access directly the variable where it is stored, *i.e.* we must pass a pointer to the variable.

[1] See *e. g. https://en.wikipedia.org/wiki/Grace_Hopper*

[2] See *e. g. https://en.wikipedia.org/wiki/C_(programming_language)*

4.2 Motivation

We start with the very simple problem of summing two integers, using a function.

```c
// An elementary example of a function to sum two integers
#include <stdio.h>

int sum(int i,int j){return i+j;}

int main (){
  int i=6,j=5;
  printf (" %d + %d = %d\n",i,j,sum(i,j));
  return 0;
}
```

Pointers/0_sum.c

This works since we need to output only the sum of two variables. How should we proceed to calculate the sum and also the product of two variables within the same function? As a function can return only one value, we need to use another approach: the variables to be modified are passed as parameters or arguments to the function. Since the function does not need to return the results, we can make it return 0 in case of success.

It would look reasonable to modify the code as follows:

```c
// A naive attempt to calculate the sum and the product of two integers using one function
#include <stdio.h>
int sumprod (int i, int j, int s, int p)
{
  s=i+j;
  p=i*j;
  printf("From sumprod: %d+%d=%d, %d*%d=%d\n",i,j,s,i,j,p);
  return 0;
}
int main()
{
  int i=6,j=5,s,p;
  if (!sumprod(i,j,s,p)) printf("Successful call to sumprod\n");
  printf("From main: %d+%d=%d, %d*%d=%d\n",i,j,s,i,j,p);
  return 0;
}
```

Pointers/1_sumprod_wrong.c

Unfortunately, this does not work as the value of `s` and `p` printed after calling `sumprod` are completely wrong. To find out why, we need to investigate in detail what is happening inside the function, by examining the addresses of the variables.

The address in memory of a variable is given by its name preceded by the symbol & (ampersand). Since the address is obviously a null or positive integer, it is encoded in `unsigned int`. The format to print the value of a pointer in decimal notation is %u, and %p in hexadecimal notation.

```c
// & is the unary operator giving the address of the variable it is applied to
// %u is the format to print an address in decimal form
#include <stdio.h>
int sumprod (int i, int j, int s, int p)
{
  printf ("Inside sumprod, address of i = %u \n",&i);
  printf ("Inside sumprod, address of j = %u \n",&j);
  s=i+j;
  p=i*j;
  printf("From sumprod: %d+%d=%d, %d*%d=%d\n",i,j,s,i,j,p);
  return 0;
}
int main()
{
  int i=6,j=5,s,p;
  printf ("From main, address of i = %u \n",&i);
  printf ("From main, address of j = %u \n",&j);
  printf("From main: %d+%d=%d, %d*%d=%d\n",i,j,s,i,j,p);
  if (!sumprod(i,j,s,p)) printf("Successful call to sumprod\n");
  printf("From main: %d+%d=%d, %d*%d=%d\n",i,j,s,i,j,p);
  return 0;
}
```

Pointers/2_sumprod_wrong_explained.c

We observe that

- the variable `s` (resp. `p`) in the main program and the variable `s` (resp. `p`) in the function `sumprod` have different addresses!
- the value of `i` (resp. `j`) inside the function `sumprod` has initially the same value as `i` (resp. `j`) in the main program.

We hence conclude that

- the variable `i` (resp. `j,s,p`) inside the function `sumprod` is a copy of the variable `i` (resp. `j,s,p`) inside the main program;
- the variables `s` and `p` in the main program are not modified by the function `sumprod`.

> In C, a parameter to a function is a copy of the variable in the program calling the function.

QUESTION: how to modify the value of a parameter to a function so that the calling function can recover the modified value?

ANSWER: use a **pointer**.

4.3 Pointer to a variable

A `pointer` is a variable capable of containing the address of another variable.
A pointer `pp` to a variable of type `type` can be declared as: `type *pp`.
For instance, the pointer `pp` to an integer can be defined by: `int *pp;`.
The variable to which a pointer points is given by the unary **deferencing operator** *, in other words `*pp` is the value of the variable to which `pp` points.[3]

The following program shows how to pass a variable to a function and modify it so that the change persists outside the function.

```
1  // To modify a variable by a function, pass its address as an argument.
2  // The corresponding argument of the function must be a pointer to that variable.
3  #include <stdio.h>
4  int sumprod_p(int i, int j, int *ps, int *pp)
5  {
6    *ps = i + j;
7    *pp = i * j;
8    printf("From sumprod_p: %d+%d=%d, %d*%d=%d\n",i,j,*ps,i,j,*pp);
9    return 0;
10 }
11
12 int main()
13 {
14   int i = 5, j = 7;
15   int sum, prod;
16   sumprod_p(i,j,&sum,&prod);
17   printf("From main: %d+%d=%d, %d*%d=%d\n",i,j,sum,i,j,prod);
18   return 0;
19 }
```

Pointers/3_sumprod_address.c

Explanation :

L16 The address `&sum` (resp. `&prod`) of the variable `sum` (resp. `prod`) is passed as an argument to the function.

L4 The corresponding parameters of the function are declared as pointers of type `int`. In this code, the pointer `ps` (resp. `pp`) contains the copy of the address of `sum` (resp. `prod`).

L6 The variable to which the pointer `ps` points is given by `*ps`, in this case it is the variable `sum` defined in the calling function, whose value is hence modified to `i + j`.

L7 The variable to which the pointer `pp` points is given by `*pp`, in this case it is the variable `prod` defined in the calling function, whose value is modified to `i * j`.

The reason why the arguments of the function `scanf` are preceded by & is now clear: since they obviously need to be modified by the function, it is necessary to pass their address.

To make a pointer points to nothing, the assignment value `NULL` should be used.

[3]This is the reason why the alternative form of declaration `type* pp` is not used in these lecture notes.

4.4 Pointer to a function

Pointers can be extremely powerful when used with functions, making these more flexible. A pointer to a function is declared with the type of the function followed by the name of the pointer preceded by $*$ between parenthesis, then the list of the function argument types between parentheses.

```c
// Pointer to a fonction of type double with one variable of type double
#include <stdio.h>
#include <math.h>
double f1(double x){return (sin(x*M_PI));}
double f2(double x){return (x*x);}
int main()
{
  int choice;
  double fx,x;
  double (*pfunc)(double); // pfunc is a pointer to a fonction of type double
                           // with one variable of type double
  printf("Enter your choice of function, 1 or 2 : ");
  scanf ("%d",&choice);
  if (choice == 1) pfunc = f1;
  else if (choice == 2) pfunc = f2;
  else {printf("Invalid choice of function\n");return 1;}
  printf("Enter value of variable x: ");
  scanf("%lf",&x);
  fx = pfunc(x);
  printf("pfunc(%lf)=%lf\n",x,fx);
  return 0;
}
```

Pointers/4_pointer_to_function.c

```c
// Pointer to a fonction of type double with two variables of type double
#include <stdio.h>
#include <math.h>
double f1(double x,double y){return sin(x*M_PI)+y;}
double f2(double x,double y){return x*x+y*y;}
int main()
{
  int choice;
  double fxy,x,y;
  double (*pfunc)(double,double); // pfunc is a pointer to a fonction of type double
                                  // with two arguments of type double
  printf("Enter your choice of function, 1 or 2 : ");
  scanf ("%d",&choice);
  if (choice == 1) pfunc = f1;
  else if (choice == 2) pfunc = f2;
  else {printf("Invalid choice of function\n");return 1;}
  printf("Enter values of variables x and y: ");
  scanf("%lf %lf",&x,&y);
  fxy = pfunc(x,y);
  printf("pfunc(%lf,%lf)=%lf\n",x,y,fxy);
  return 0;
}
```

Pointers/5_pointer_to_function2.c

The full power of pointers to functions become even more obvious when they are passed as parameters to other functions. The pointer is declared in the list of parameters to the function in a way identical to the declaration in a code. The actual parameters (not their types) of the pointed function must be passed separately as parameters of the function.

```c
// Pointer to a function passed as an argument to a function
// In the calling sequence, the name of the function or a pointer
// to the function can be used in the list of arguments
#include <stdio.h>
#include <math.h>
double f2(double(*f)(double),double x,double y)
{
  return(f(x*M_PI)+y);
}

int main(void)
{
  double x,y;
  printf("Calculating cos(x*M_PI)+y, enter x and y: ");
  scanf("%lf %lf",&x,&y);
  printf("cos(%lf*M_PI)+%lf =%lf\n",x,y,f2(cos,x,y));
  printf("Calculating sin(x*M_PI)+y:\n");
  printf("sin(%lf*M_PI)+%lf = %lf\n",x,y,f2(sin,x,y));
  return 0;
}
```

Pointers/6_function_as_arg.c

```c
// Pointer to a two variable function passed as an argument to a function
// atan2(y,x) returns the angle theta in radian with y=sin(theta) and x=cos(theta)
#include <stdio.h>
#include <math.h>
double f2(double s,double t,int k,double(*f)(double,double))
{
  double fct=f(t,s);
  printf("From f2: f(%lf,%lf)=%lf\n",t,s,fct);
  return(fct+k);
}

int main(void)
{
  int i;
  double x,y;
  printf("Please enter integer i: ");
  scanf("%d",&i);
  printf("Please enter double x and y: ");
  scanf("%lf %lf",&x,&y);
  printf("atan2(%lf,%lf) + %d = %lf\n",y,x,i,f2(x,y,i,atan2));
  return 0;
}
```

Pointers/7_function_as_arg2.c

4.5 Double pointers

A pointer that points to a pointer that points to a variable is called a double pointer. In the case of a variable of type `int`, a double pointer can be declared as: `int **pp_i` as in the example below.

```c
// Double pointers
#include <stdio.h>
int main()
{
  int m=42,n=666;
  int *p_i, **pp_i;
  p_i = &m;
  pp_i = &p_i;
  printf(" &m = %p m = %d\n",&m,m);
  printf(" &n = %p n = %d\n",&n,n);
  printf(" &p_i = %p p_i = %p *p_i = %d\n",&p_i, p_i,*p_i);
  printf(" pp_i = %p *pp_i = %p **pp_i = %d\n",pp_i,*pp_i,**pp_i);
  p_i = &n;
  printf(" &p_i = %p p_i = %p *p_i = %d\n",&p_i,p_i,*p_i);
  printf(" pp_i = %p *pp_i = %p **pp_i = %d\n",pp_i,*pp_i,**pp_i);
  printf(" &pp_i = %p\n",&pp_i);
  return 0;
}
```

Pointers/8_double_pointer.c

NB: A pointer can be initialized by assigning the address of another variable.

4.6 Good practice in naming pointers

Since the first letter of a pointer's name can be `p`, it is better to adopt the convention that the first two letters of a pointer's name are `p_`.

If the name of a pointer starts with the letters `p_`, it is clear that

1. it must be initialised using the address of another variable;
2. when preceded by `*`, its value is that of a variable, that can typically be used for arithmetic, logical or string operations.

Similarly, if the name of a double pointer starts with the letters `pp_`, it is clear that

1. it must be initialised using the address of another pointer;
2. when preceded by `*`, its value must be a pointer;
3. when preceded by `**`, its value must be that of a variable.

This is illustrated in `8_double_pointer.c`.

Double pointers are extremely useful to define matrices, presented in Chapter 6. The above convention to name pointers reduces the risk of errors.

4.7 Summary

In this Chapter, we have seen that

1. A pointer is variable that can hold the address of another variable or has the value `NULL` if it points to nothing.
2. A pointer `p_i` to a variable of type `type` is declared using the command: `type *p_i`.
3. A pointer can be initialized by assigning the address of another variable, which is given by its name preceded by the unary operator &.
4. Pointers are necessary to modify variables in a function and to pass them back to the calling function since in C, parameters to a function are always transferred by copy of value.
5. A pointer `p_f` to a function `f(x)` is declared as: `type (*p_f) (type)` where the first `type` refers to the type of variable returned by the function `f` and the second refers to the type of the argument `x`. These two types may of course be different.
6. It is a good practice to use names of pointers starting with `p_`, of double pointers starting with `pp_`, etc.

4.8 Exercices

1. Write a code that asks for 3 reals a, b, c and using a function `rroot`, calculates the number `n` and values of the different real solutions `xp` and `xm` of $ax^2 + bx + c = 0$, with

$$\texttt{xp}, \texttt{xm} = \frac{-b' \pm \sqrt{b'^2 - ac}}{a}$$

   where $b' = b/2$. If the discriminant $\sqrt{b'^2 - ac}$ is 0, the roots are degenerated: `xp` = `xm` and $n = 1$. For $\sqrt{b'^2 - ac} < 0$, there is no real solution: $n = 0$. The `main` code (not the function `rroot`) must print the value of n and n solutions.

2. Modify `7_function_as_arg.c` in order to use the pointer `fpt` first to the function `cos` then to the function `sin`.

3. Write a code that calculate the square of a function `f(x)` using a function `double x2(double x)` that returns the square of `x`. Test the code with

   (a) `x=5.0` for `x2`

   (b) `x=3.0` and `f=sqrt`

   (c) `x=M_PI/4` and `f=cos`

4. Newton-Raphson method for root search

   (a) Write a program to determine the root of a function $f(x)$ using the Newton-Raphson method from the starting point $x^{(0)}$

$$x^{(i+1)} = x^{(i)} - \frac{f\left(x^{(i)}\right)}{f'\left(x^{(i)}\right)} \qquad \text{for } i = 1, 2, \ldots$$

   Print the number of iterations needed to converge to machine accuracy.

   (b) Apply the code to calculate the square root of a real number, using different starting values.

   (c) Expand this code to calculate the roots of the Legendre polynomial of order n, $P_n(x)$, which must be calculated using the stable recurrence relation

$$nP_n(x) = (2n-1)xP_{n-1}(x) - (n-1)P_{n-2}(x)$$

   with $P_0 = 1$, $P_1 = x$, while the derivative can easily be obtained with the relation

$$(1 - x^2)P'_n(x) = nP_{n-1}(x) - nxP_n(x).$$

   Take the starting point for the k^{th} root as

$$x_k^{(0)} = \cos\left(\frac{(4k-1)\pi}{4n+2}\right) \qquad \text{for } k = 1, \ldots, n.$$

Chapter 5

Arrays

5.1 Introduction

Arrays are convenient to handle large number of data in an efficient way. The two main advantages are spatial locality and compact notation similar to that used in mathematics.

In this chapter, we show that the simplest way – automatic, on the stack – to declare arrays should be limited to small dimensions in order to avoid runtime stack overflow. In C, the concept of array is closely related to that of pointer, due to array-pointer interchangeability in expressions. The array notation with brackets is a `Syntactic Sugar`[1] whose sole function is to make the code easier to read by a human user. There are thus two different and totally equivalent syntaxes to handle arrays. It is very important to master both equally well in order to be able to develop clear and efficient codes and to understand other people's codes. The last sections of this chapter are devoted to dynamical allocation – on the heap – of arrays, with the full use of pointers, demonstrating again their absolute need in C programming.

5.2 Simple declaration of an array

```c
// A very basic example handling an array of doubles
#include <stdio.h>
#include <math.h>
int main()
{
  int i;
  double V[10];
  for (i=0;i<10;i++) V[i] = sqrt((double)i);
  for (i=0;i<10;i++) printf("V[%d]=%15.8lf\n",i,V[i]);
  return 0;
}
```

Array/1_basic_array.c

[1] The Mechanical evaluation of expressions, P. J. Landin, Comput J (1964) 6 (4): 308-320

5.3 A basic way to initialize an array

```c
// Explicit initialization of an array
#include <stdio.h>
#define N 5
int main()
{
  int i;
  int V[N]={1492,1648,1789,1969,2017};
  for (i=0;i<N;i++) printf("V[%d]=%d\n",i,V[i]);
  return 0;
}
```

Array/2_init_array.c

5.4 Automatic declaration of an array with size defined in runtime

```c
#include <stdio.h>
#include <math.h>
int main()
{
  int i,m;
  printf ("Enter dimension of array V: ");
  scanf("%d",&m);
  double V[m];
  for (i=0;i<m;i++) V[i] = sqrt((double)i);
  for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,V[i]);
  return 0;
}
```

Array/3_automatic_array.c

5.5 Problem of automatic declaration of arrays

Arrays declared automatically as in the examples above are on the `stack`. This part of the memory is fast but its size rather limited, leading to a `segmentation fault` when too many data are used. This is called a `stack overflow`. In Linux, the size of the stack can be found using the command `ulimit -a` or `ulimit -s`. For example:

```
> ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
file size               (blocks, -f) unlimited
max locked memory       (kbytes, -l) unlimited
max memory size         (kbytes, -m) unlimited
open files                      (-n) 256
pipe size            (512 bytes, -p) 1
stack size              (kbytes, -s) 32768
```

```
cpu time               (seconds, -t) unlimited
max user processes             (-u) 709
virtual memory          (kbytes, -v) unlimited
```

As suggested by its name, the stack is filled on the basis of *first come, first placed*. It is automatically emptied when leaving the function, implying that its variables can no longer be accessed, except by accident, for instance when a pointer wanders, following a bug, outside the code memory segment.

The size of the stack can be output from a code in C as shown in `4_stack_size.c`. In this example, when the size of the array `V` is equal or larger than 4193668, *i.e.* the largest index is equal or larger than 4193667, a segmentation fault occurs.

```c
// This code demonstrates that automatic allocation of an array is limited by the stack size
// For Stack limit = 33554432, the maximum size of an array of doubles in THIS PARTICULAR CODE is 4193667
// Check what happens for larger values of m
// In Linux: ulimit -s gives the maximum stack size in kbytes
//           ulimit -a reports all current limits

#include <stdio.h>
#include <math.h>
#include <sys/resource.h>
int main()
{
  struct rlimit limit;
  getrlimit (RLIMIT_STACK, &limit);
  printf ("\nStack limits:\n");
  printf ("Current (soft) limit= %ld (bytes)\nMaximum (hard) limit= %ld (bytes)\n", limit.rlim_cur, limit.rlim_max);
  printf ("Corresponding number of doubles: %d %d\n",limit.rlim_cur/8,limit.rlim_max/8);
  int i,m;
  printf ("\nEnter dimension of array V: ");
  scanf("%d",&m);
  double V[m];

  printf("size of V: %d\n",(int)sizeof(V));
  printf("size of structure limit: %d\n",(int)sizeof(limit));
  printf("size of i: %d\n",(int)sizeof(i));

  for (i=0;i<m;i++) V[i] = sqrt((double)i);
  printf("    V[%2d]   =%15.8lf\n",m-1,V[m-1]);
  printf("pow(V[%2d],2)=%15.1lf\n",m-1,V[m-1]*V[m-1]);
  return 0;
}
```

Array/4_stack_size.c

`sizeof` is an intrinsic function that returns the size of its argument in bytes.

5.6 Interchangeability of `V[i]` and `*(V+i)`

```c
// This program shows the relation between arrays and pointers:
// in an expression (not in a declaration), V[i] is equivalent to *(V+i)
// This array-pointer interchangeability is an example of Syntactic Sugar
#include <stdio.h>
#include <math.h>
int main()
{
  int i,m;
  printf ("Enter dimension of array V: ");
  scanf("%d",&m);
  double V[m];
  for (i=0;i<m;i++) *(V+i) = sqrt((double)i);
  for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,V[i]);
  return 0;
}
```

Array/5_array_pointer.c

5.7 Pointer to an array

```c
// Since in an expression, V[i] is a syntactic sugar of *(V+i),
// V is a pointer to the first element of the array V, V[0]
// Hence V can be used to initialize another pointer p,
// p[i] is equivalent to *(p+i), which is the value of the ith variable
// in memory after the address of p
#include <stdio.h>
#include <math.h>
int main()
{
  int i,m;
  printf ("Enter dimension of array V: ");
  scanf("%d",&m);
  double V[m];
  for (i=0;i<m;i++) V[i] = sqrt((double)i);

  double *p;
  p = V; // equivalent to: p = &V[0];
  for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,p[i]);
  // The above line is equivalent to
  for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,*(p+i));
  return 0;
}
```

Array/6_pointer_array.c

5.8 Array as argument to a function

```c
// An example of how to pass an array to a function
#include <stdio.h>
void def_print_v(int V[],int m){
  int i;
  printf ("\nFrom def_print_v:\n");
  for (i=0;i<m;i++) {
    V[i] = i;
    printf("V[%d]=%d\n",i,V[i]);}
}
int main(){
  int i;
  int m=10;
  int V[m];
  printf ("\nFrom main before def_print_v:\n");
  for (i=0;i<m;i++) printf("V[%d]=%d\n",i,V[i]);
  def_print_v(V,m);
  printf ("\nFrom main after def_print_v:\n");
  for (i=0;i<m;i++) printf("V[%d]=%d\n",i,V[i]);
  return 0;
}
```

Array/7_function_print_v.c

5.9 Another method to pass an array to a function

```c
// Another way of passing an array to a function
#include <stdio.h>
void def_print_v(int *V,int m){
  int i;
  printf ("\nFrom def_print_v:\n");
  for (i=0;i<m;i++) {
    V[i] = i;  // V[i] can be replaced by *(V+i)
    printf("V[%d]=%d\n",i,V[i]); // V[i] can be replaced by *(V+i)}
  }
}

int main(){
  int i,m;
  printf ("Enter dimension of array V: ");
  scanf("%d",&m);
  int V[m]; // Here, V[m] CANNOT be replaced by *(V+m)
  printf ("\nFrom main before def_print_v:\n");
  for (i=0;i<m;i++) printf("V[%d]=%d\n",i,V[i]);
  def_print_v(V,m);
  printf ("\nFrom main after def_print_v:\n");
  for (i=0;i<m;i++) printf("V[%d]=%d\n",i,V[i]);
  return 0;
}
```

Array/8_function_print_v_version2.c

5.10 Difference between array and pointer

```c
// V[i] is a syntactic sugar of *(V+i)
// V is a pointer to the first element of the array V, V[0]
// However, a pointer and an array are not the same things, the main
// difference is in the declaration
#include <stdio.h>
#include <math.h>
int main()
{
  int i,m;
  printf ("Enter dimension of array V: ");
  scanf("%d",&m);
  double V[m]; // Here, the memory for m consecutive doubles is ALSO reserved
               // and V points to the first double
  for (i=0;i<m;i++) V[i] = sqrt((double)i);

  double *p; // Here, a pointer to a double is declared

  int test;
  printf ("To observe the runtime error, enter 0, otherwise enter 1: ");
  scanf ("%d",&test);
  // The following instruction gives rise to a runtime error since
  // p points to nothing
  if (test == 0) for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,*(p+i));

  p = V;      // p must be initialized by giving the address of an ALREADY
              // existing double
  for (i=0;i<m;i++) printf("V[%2d]=%15.8lf\n",i,*(p+i));
  return 0;
}
```

Array/9_array_pointer_difference.c

5.11 Dynamical allocation of an array

To overcome the problem of stack size, arrays must be allocated `dynamically` – on the `heap` – using `malloc` or `calloc`. Including `<stdlib.h>` is necessary to define these functions.

It is a good practise to convert the type of the value returned by `malloc` and `calloc` to the type of pointer required, in this example (double*). Memory allocated should always be freed when the array is no longer used, and in any case before the end of the run. **Not applying these good practises may cause runtime errors that are difficult to localize and correct.**

```
// Dynamical allocation of an array of doubles
// Maximum size of V = 2,147,483,647 = pow(2,31)-1
// Compare with 4_stack_size.c where maximum size of V = 4,193,667 < pow(2,22)
#include <stdio.h>
#include <stdlib.h>
int main(){
  int n;
  double *V;
  printf ("Enter length of array V: ");
  scanf ("%d",&n);
  V=(double*)malloc(n*sizeof(double));
  V[1] = (double)1;
  V[n-1] = (double)(n-1);
  printf("V[%d]=%lf\n",1,V[1]);
  printf("V[%d]=%lf\n",n-1,V[n-1]);
  free(V);
  return 0;}
```

Array/10_double_array_malloc.c

The size of the array is limited by the representation of integers in 32 bits. This is demonstrated in the code below, where using unsigned integers for the array indices increases its largest possible size by more than a factor 2.

```
// Dynamical allocation of an array of doubles
// Unsigned integers are represented on 32 bits
// Maximum size of V = 4,294,967,295 = pow(2,32)-1
// Compare with 10_double_array_alloc.c where maximum size of V is
// 2,147,483,647 = pow(2,31)-1
#include <stdio.h>
#include <stdlib.h>
int main(){
  unsigned int n;
  double *V;
  printf ("Enter length of array V: ");
  scanf ("%u",&n);
  V = (double*)malloc(n*sizeof(double));
  V[n-1] = (double)(n-1);
  printf("V[%u]=%lf\n",n-1,V[n-1]);
  free (V);
  return 0;}
```

Array/11_double_array_malloc_unsigned_index.c

The function `calloc` allocates the array and initializes all the elements to 0.

```c
// Dynamical allocation and initialization to zero of an array of doubles using calloc
// NB: Elements of an array allocated using malloc might incidentally be initialized to zero
#include <stdio.h>
#include <stdlib.h>
int main(){
  int i,n;
  double *V;
  printf ("Enter length of array V: ");
  scanf ("%d",&n);

  V=(double*)malloc(n*sizeof(double));
  for (i=0;i<n;i++) if (V[i] != 0) printf("After malloc: V[%d]=%lf\n",i,V[i]);
  free (V);

  V=(double*)calloc(n,sizeof(double));
  for (i=0;i<n;i++) if (V[i] != 0) printf("After calloc: V[%d]=%lf\n",i,V[i]);
  free(V);

  return 0;
}
```

Array/12_double_array_calloc.c

5.12 Allocating and using an array in independent functions

In professional codes where the number of arrays to handle simultaneously can be very large, it is frequent to allocate and initialize arrays in dedicated functions. We consider the case where

1. the dimension `n` of the array `V` is read in the `main` function,
2. the array is allocated on the heap in function `array_init`,
3. the elements are printed in function `array_print`.
4. the functions `array_init` and `array_print` are called by the `main`.

Since **the arguments of a function are copies of the values of the variables from the calling function**, it is necessary to use a pointer to the address of the array in order to modify it using the function `malloc` or `calloc` inside `array_init`. Since in an expression, an array is equivalent to a pointer, a double pointer is needed.

```c
// How to pass an array allocated in a function to the calling function
#include <stdio.h>
#include <stdlib.h>

int array_init (int **V,int size)
{
  int i;
  *V=malloc(size*sizeof(int));
  for (i=0;i<size;i++) (*V)[i]=i;
  return 0;
}

void array_print (int **t,int size)
{
  int i;
  printf("array_print:\n");
  for (i=0;i<size;i++) {
    printf(" t(%d) = %d",i,(*t)[i]);
    printf(" t(%d) = %d",i,*(*t+i)); // alternative form
    printf("\n");}
}

int main()
{
  int n;
  int *V;

  printf("Please enter number of elements of array: ");
  scanf("%d",&n);

  array_init (&V,n);
  array_print (&V,n);
  free (V);

  return 0;
}
```

Array/13_array_alloc_in_function.c

In the following counter-example, we show the errors generated by a code that attempts to create and modify an array without the correct use of pointers.

`printf("%5d %9.5lf\n",i,A[i]);`

gives rise to a segmentation fault.

```c
// Demonstrating by a counter-example the importance of remembering that in C:
// 1. parameters to a function are always transferred by copy of values
// 2. a pointer must be used in order to modify and transfer back parameters
//    from within a function
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MIN -5.0
#define MAX 5.0
double norm = (MAX - MIN)/(double)RAND_MAX;

void array_rand (int *N,double *A)
{
  int i;
  printf("Printing from array_rand\n");
  printf("&A = %p\n",&A);
  printf ("Dynamical allocation of an array A[N] of random doubles between %3.1lf and %3.1lf
    \n",MIN,MAX);
  printf ("Please enter N: ");
  scanf ("%d",N);
  printf("N = %d &N = %p\n",*N,N);
  //  double A[N]; compilation error : 'A' redeclared as different kind of symbol
  A = malloc (*N*sizeof(double));
  srand(time(NULL));
  for (i = 0; i < *N; i++) {
    A[i] = MIN + rand()*norm;
    printf("A[%d]=%9.5lf  &A[%d]=%p\n",i,A[i],i,&A[i]);
  }
}

int main()
{
  int i,N;
  double *A;
  printf("Printing from main:\n N = %d &N = %p\n",N,&N);
  printf("&A = %p\n",&A);
  // printf("A[1] = %lf\n",A[1]); //Segmentation fault
  array_rand (&N,A);
  printf("Printing from main: N = %d &N = %p\n",N,&N);
  for (i = 0; i < N; i++) printf("&A[%d] = %p\n",i,&A[i]);
  printf("Print array A\n");
  for (i = 0; i < N; i++) printf("%5d %9.5lf\n",i,A[i]);
  return 0;
}
```

Array/14_array_alloc_in_function_wrong.c

5.13 Summary

In this chapter, two methods of defining and handling arrays have been presented:

1. Sections 2–4: on the stack, using automatic declaration. The array is allocated using the type followed by the name of the array with the dimension between square brackets ;
2. Section 11: on the heap, using `malloc` and `calloc`.

In section 5, we show that for large arrays, limitation of stack size leads to stack overflow and segmentation fault. Since the stack is emptied only when leaving a function, a similar problem will occur as soon as the total size of arrays used within the same function leads to stack overflow.

In section 6, we show the important concept of interchangeability between array and pointer:

in an expression, `V[i]` is equivalent to `*(V+i)`

Both notation can thus be used indifferently and **both must be mastered**, since

- the notation with square brackets is closer to the mathematical notation and might be preferred by scientists as Syntactic Sugar ;
- the notation with the pointer deference operator * clearly shows that consecutive elements of an array are adjacent in memory, a concept of huge importance in High Performance Computing.

Array-pointer interchangeability is further exploited in section 7 to initialize pointer to array. This is important for passing array as argument to a function, as shown in sections 8 and 9. In section 10, we draw attention to the fact that array-pointer interchangeability does not imply that arrays and pointers are equivalent: an important difference occurs in declaration.

Finally in section 12, we show how a pointer to an array, that is a double pointer, must be used to initialize and transfer arrays between functions.

Another natural use of double pointers will be presented in the next chapter on **matrices**.

5.14 Memo

Ce qu'il faut retenir des pointeurs

Toute variable est caractérisée par une et une seule adresse dans la mémoire.
L'adresse d'une variable est donnée par son nom précédé de &.
On imprime la valeur d'un pointeur avec le format %p.

Une variable de type pointeur contient l'adresse d'une donnée ainsi que son type.

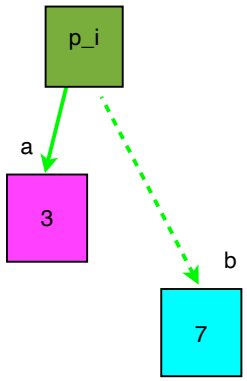
Une variable p_i de type pointeur d'entier se déclare par : int *p_i;

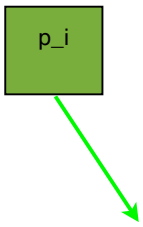
Supposons que l'on ait au préalable déclaré et initialisé deux entiers : int a = 3, b = 7;

On peut faire pointer p_i vers a en attribuant à p_i l'adresse de a : p_i = &a;

La valeur de la donnée pointée par p_i est notée *p_i, elle vaut ici 3.

On peut ensuite faire pointer p_i vers un autre entier, par exemple b : p_i= &b;
*p_i vaut alors 7.





Supposons que l'on déclare un tableau : int c[10];

Si p_i pointe vers le premier élément de c : p_i = &c[0];

alors p_i = 12530 et *p_i = -5 = c[0] = *c

Puisque sizeof(int) vaut 4, on a

p_i + 1 = 12534 et *(p_i+1) = -7 = c[1] = *(c+1)

adresse	12530	12534	12538	12542	12546	12550	12554	12558	12562	12566
valeur	-5	-7	-9	-6	-3	0	1	3	5	11
	c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]

Attention : si on force p_i à pointer vers une donnée dont le type est différent de int, par exemple p_i = (int *)&z avec z déclaré de type double, *p_i interprétera les 4 bytes à l'adresse de z comme ceux d'un entier, ce qui conduira certainement à un résultat non voulu.
Exemple : cf IEEE754.c

31	133	235	81	184	30	9	64
1 374 389 535				1 074 339 512			
3.14							

```c
// This code implements the examples presented in pointers_memo.pdf
#include <stdio.h>
int main(){
  int a=3,b=7,i;
  int *iptr;
  int c[]={-5,-7,-9,-6,-3,0,1,3,5,11};
  double z = 3.14;

  iptr = &a;
  printf("%d\n",*iptr);
  iptr = &b;
  printf("%d\n",*iptr);

  for (iptr=c,i=0;i<10;iptr++,i++) printf("%d\n",*iptr);

  iptr = (int*)&z;
  printf("%d\n",*iptr);
  ++iptr;
  printf("%d\n",*iptr);

  return 0;
}
```

Array/IEEE754.c

5.15 Exercices

1. Compile the following code and analyse the message from the compiler. Run the following code for increasing values of array length `N`. The outputs are wrong for `N`> 27. Find out why and correct the code.

```c
// Question: Why is this code wrong? Try it with N=28 or bigger.
#include <stdio.h>
#include <stdlib.h>
int* create_array(int dim){
  int i,x[dim];
  for (i=0;i<dim;i++) x[i]=i;
  return x;
}
int main(){
  int i,N,*v;
  printf("Please enter dimension of array v: ");
  scanf("%d",&N);
  v=create_array(N);
  for (i=0;i<N;i++) printf(" v[%d]=%d\n",i,v[i]);
  return 0;
}
```

Array/warning.c

2. Write a code using the sieve of Erastosthenes to determine all prime numbers up to a given integer `M`. To do this,

   1. create an array of integers `X[M+1]` and set all the elements to 0, except `X[0]` and `X[1]` which are obviously not prime.
   2. For `i`=2,3,4,..., not exceeding $\sqrt{\texttt{M}}$
      if `X[i]` is false (that is untrue)
      for `j`=$\texttt{i}^2, \texttt{i}^2+\texttt{i}, \texttt{i}^2+2\texttt{i}, \ldots$ not exceeding `M`
      set `X[j]`=1.

   At the end of the algorithm, all `i` such that `X[i]`=0 are prime. Print also the number of primes found.

Chapter 6

Matrices

6.1 Introduction

Most equations in fundamental science can be conveniently written in compact, matrix form. Many numerical methods can be decomposed into a succession of simple algebraic operations such as matrix addition, multiplication, inversion, diagonalization, etc ... Libraries such as `BLAS` (Basic Linear Algebra Subprograms) and `LAPACK` (Linear Algebra PACKage)[1] are widely used in High Performance Computing, with the subset `LINPACK` defining the standard to measure the performance of the world TOP500 supercomputers [2]. It is hence very important to learn how to declare and manipulate matrices efficiently in the C programming language too.

6.2 Automatic declaration of a 2D matrix

In C, matrices are defined using pointers: a 2D matrix `A[m][n]` is an array `A[m]` of `m` pointers, with `A[i]` pointing to the first element of the i^{th} row of `n` elements `A[i][j]`.

```c
// A simple code for the automatic allocation of a 2D matrix
#include <stdio.h>
#define nrow 3
#define ncol 4
int main(){
  int t[nrow][ncol];
  int i,j,a=0;
  for (i=0;i<nrow;i++)
    for (j=0;j<ncol;j++){t[i][j]=a;a++;}
  printf("Printing matrix elements of t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) printf("%3d",t[i][j]);
    printf("\n");}
  return 0;}
```

Matrix/1_mat2d_automatic.c

[1] http://www.netlib.org/

[2] https://www.top500.org/

Since the elements of an array are numbered from 0, so are the rows and columns of a matrix. In other words, the elements of a matrix `A[m][n]` are

```
A[0][0]     A[0][1]   ...     A[0][n-1]
A[1][0]     A[1][1]   ...     A[1][n-1]
   ...         ...    ...        ...
A[m-1][0]   A[m-1][1]  ...   A[m-1][n-1]
```

Since matrices are arrays of pointers, they inherit all the properties of pointers described in Chapter 2, in particular the interchangeability between `V[i]` and `*(V+i)`.

```c
// This code demonstrates again the interchangeability of [ ] and *
// t[i][j] is syntaxically equivalent to *(*t+i*ncol+j)
//                                  and to *(*(t+i)+j))
#include <stdio.h>
#define nrow 3
#define ncol 4
int main()
{
  int t[nrow][ncol];
  int i,j,a;

  a=0;
  for (i=0;i<nrow;i++)
    for (j=0;j<ncol;j++){
      t[i][j]=a;
      a++;
    }

  printf("Printing matrix elements of t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) printf("%3d",*(*t+i*ncol+j));
    printf("\n");
  }

  printf("t[0][0] = **t      = %4d\n",**t);
  printf("t[0][2] = *(*t+2) = %4d\n",*(*t+2));
  printf("t[1][0] = *(*t+4) = %4d\n",*(*t+4));
  printf("t[1][0] = *(*(t+1)) = %4d\n",*(*(t+1)));
  printf("t[1][3] = *(*t+7) = %4d\n",*(*t+7));
  printf("t[1][3] = *(*(t+1)+3) = %4d\n",*(*(t+1)+3));

  return 0;
}
```

Matrix/2_mat2d_ptr.c

t[i][j] is a syntactic sugar of *(*t+i*ncol+j)

The knowledge of the number of columns is hence essential in order to manipulate the elements of a 2D matrix. The number of rows must of course be known in order not to trepass the boundaries of the matrix but since there is no verification, it is not required in the same way as for the number of columns. This is of crucial importance for the transfer of matrices to a function (*cf.* next section).

Another consequence is that matrices can be manipulated using only pointers, which can lead to shorthand but, for the non-initiated, somehow obscure lines of codes, as demonstrated in the following two examples.

```c
// Demonstrating that a 2D matrix is an array of pointers, each of which points to a row
#include <stdio.h>
#define nrow 3
#define ncol 4
int main()
{
  int t[nrow][ncol];
  int i,j,a;
  int *p;
  for (i=0,a=0;i<nrow;i++)
    for (j=0;j<ncol;j++){t[i][j]=a;a++;}
  printf("Printing matrix elements of t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (p=t[i],j=0;j<ncol;p++,j++) printf("t[%d][%d]=%3d  ",i,j,*p);
    printf("\n");}
  printf("Printing matrix elements of t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (p=t[i],j=0;j<ncol;j++) printf("t[%d][%d]=%3d  ",i,j,p[j]);
    printf("\n");}
  return 0;
}
```

Matrix/3_mat2d_ptr2.c

```c
// A more graphical demonstration that a 2D matrix is an array of pointers,
// each of which points to a row
#include <stdio.h>
#define nrow 3
#define ncol 4
int main()
{
  int t[nrow][ncol];
  int i,j,a;
  int *p;
  for (i=0,a=0;i<nrow;i++)
    for (j=0;j<ncol;j++){t[i][j]=a;a++;}
  for (i=0;i<nrow;i++) printf("t[%d]=%u\n",i,t[i]);
  printf("\nPrint matrix t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) printf("t[%d][%d]=%3d  ",i,j,t[i][j]);
    printf("\n");}
  printf("\nPrint matrix t[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) {
      p=t[i];
      printf("t[%d][%d]=%3d  ",i,j,p[j]);}
    printf("\n");}
  return 0;
}
```

Matrix/4_mat2d_ptr3.c

In some cases, it might be convenient to store data in a table with variable number of elements in each row, as shown in the example below.

```c
// How to define a table with rows of different length
#include <stdio.h>
#define nrow 3
int main()
{
  int length[nrow]={2,1,4};
  int row1[]={1,2};
  int row2[]={3};
  int row3[]={4,5,6,7};
  int *mat[]={row1,row2,row3};

  int i, j;

  for (i=0; i<nrow; i++){
    for (j=0; j<length[i];j++)
      printf ("mat[%d,%d]=%d  ",i,j,mat[i][j]);
      printf("\n");}

  return 0;
}
```

Matrix/5_irregular_table.c

6.3 Transfer of matrices to a function

A 2D matrix can be transferred in the list of parameters to a function in a way similar as for an array: the only difference involves the absolute necessity to precise the number of columns, for the reason indicated above: t[i][j] is a syntactic sugar of *(*t+i*ncol+j). The number of rows should also be transferred in order to respect the boundaries of the matrix but there is nothing to guarantee this.

```c
// An elementary code passing a 2D matrix as an argument to a function
#include <stdio.h>
#define nrow 3
#define ncol 4
void print_mat(int A[][ncol])
{
  int i,j;
  printf("Printing matrix elements of A[%d][%d]:\n",nrow,ncol);
  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) printf("%3d",A[i][j]);
    printf("\n");}
}
int main()
{
  int t[nrow][ncol];
  int i,j,a;
  for (i=0,a=0;i<nrow;i++)
    for (j=0;j<ncol;j++){t[i][j]=a;a++;}
  print_mat(t);
  return 0;
}
```

Matrix/6_mat2d_to_function.c

In this simple example,

`void print_mat(int A[][ncol])`

can be replaced by

`void print_mat(int A[nrow][ncol])`

without any consequence on the results. On the other hand, the parameter `ncol` cannot be omitted as it would give rise to the compilation error:

`Array type has incomplete element type.`

6.4 Dynamical allocation of matrices

Matrices are very useful in High Performance Computing as they guarantee a fast access to a large number of data if they are contiguous in memory. This is demonstrated in the following two examples.

Although the first method might appear cumbersome, it is the only way to guarantee that all the elements of the matrix are contiguous in memory. This is obvious as an array `ptBlock` of $m \times n$ elements is reserved in the instruction

`int* ptBlock=(int *) malloc(nrow*ncol*sizeof(int));`

while

`ppt[i]=&ptBlock[i*ncol];`

makes `ppt[i]` points to the first element of row i which is the $i \times$ `ncol`th element of the array `ptBlock`.

```c
// How to define dynamically a 2D matrix of contiguous elements
#include <stdio.h>
#include <stdlib.h>
int main()
{
  int ncol,nrow;

  printf("Defining a 2D matrix:\n");
  printf("Enter the number of rows:");
  scanf("%d",&nrow);
  printf("Enter the number of columns:");
  scanf("%d",&ncol);

  int *ptBlock=(int *) malloc(nrow*ncol*sizeof(int));
  int **ppt=(int **) malloc(nrow*sizeof(int *));
  for(int i=0;i<nrow;i++) ppt[i]=&ptBlock[i*ncol];

  printf("Please enter %d integers: ",nrow*ncol);
  for (int i=0;i<nrow;i++)
    {
      for (int j=0;j<ncol;j++) scanf("%d",&ppt[i][j]);
    }
  printf("Your matrix is:\n");

  for (int i=0;i<nrow;i++)
    {
      for (int j=0;j<ncol;j++)
        {
          printf("t[%d][%d]=%d ",i,j,ppt[i][j]);
        }
      printf("\n");
    }

  free (ppt[0]);   //  equivalent to free (ptBlock);
  free (ppt);

  return 0;
}
```

Matrix/7_mat2d_alloc.c

The second method should be used with caution and only in limited cases where the fast access to contiguous elements is not necessary. Elements of different rows might be contiguous but this would only be true accidentally as shown in the last printout of `t` using a pointer.

```c
// How to define dynamically a 2D matrix of NOT NECESSARILY contiguous elements
#include <stdio.h>
#include <stdlib.h>
int main()
{
  int ncol,nrow;
  int **t;
  int i,j;

  printf("Defining a 2D matrix:\n");
  printf("Enter the number of rows:");
  scanf("%d",&nrow);
  printf("Enter the number of columns:");
  scanf("%d",&ncol);

  t=(int **) malloc(nrow*sizeof(int *));
  for(i=0;i<nrow;i++) t[i]=(int *) malloc(ncol*sizeof(int));

  printf("Please enter %d integers: ",nrow*ncol);
  for (i=0;i<nrow;i++)
    for (j=0;j<ncol;j++)
      scanf("%d",&t[i][j]);
  printf("Your matrix is:\n");

  for (i=0;i<nrow;i++){
    for (j=0;j<ncol;j++) printf("t[%d][%d]=%d address: %p ",i,j,t[i][j],&t[i][j]);
    printf("\n");}

  int *p;
  for (p=t[0],i=0;i<nrow*ncol;p++,i++) printf("i=%d *p=%d\n",i,*p);

  free (t);

  return 0;
}
```

Matrix/8_mat2d_alloc_noncontiguous.c

The function `calloc` can be used instead of `malloc` to ensure that the elements are initialized to zero. The syntax is however slightly different (*cf.* arrays/12_double_array_calloc.c).

Since in C language, parameters to a function are always transferred by copy of values, pointers must be used in order to modify variables from within a function and to transfer the modified data to other functions. As a matrix is a pointer to an array, they are in fact double pointers as clearly shown in the declaration in `7_mat2d_alloc.c`. In order to create or modify a 2D matrix inside a function, a pointer to a double pointer, that is a triple pointer, must be used.

```c
// How to allocate and deallocate a 2D matrix with contiguous elements in a function
#include <stdio.h>
#include <stdlib.h>
void mat_make(int ***pppA,int nrow,int ncol){
  int *pA_alloc=(int *)malloc(nrow*ncol*sizeof(int));
  int **ppA=(int **)malloc(nrow*sizeof(int*));
  for(int i=0;i<nrow;i++) ppA[i]=&(pA_alloc[i*ncol]);
  *pppA = ppA;
}
void mat_fill(int **ppA,int nrow,int ncol){
  printf("Enter integer elements of 2d matrix: ");
  for(int i=0;i<nrow;i++)
    for(int j=0;j<ncol;j++) scanf("%d",&ppA[i][j]);
}
void mat_print(int **ppA,int nrow,int ncol){
  printf("2d-matrix:\n");
  for(int i=0;i<nrow;i++){
    for(int j=0;j<ncol;j++) printf("[%d][%d]=%d\t",i,j,ppA[i][j]);
    printf("\n");
  }
}
void mat_free (int **ppA)
{
  free (ppA[0]);   //  equivalent to free (pA_alloc);
  free (ppA);
}

int main(){
  int nrow,ncol,**ppA;
  printf("Give the dimensions nrow et ncol of a 2d matrix: ");
  scanf("%d %d",&nrow,&ncol);
  printf("nrow = %d, ncol = %d\n",nrow,ncol);
  mat_make(&ppA,nrow,ncol);
  mat_fill(ppA,nrow,ncol);
  mat_print(ppA,nrow,ncol);
  mat_free(ppA);
  return 0;
}
```

Matrix/9_mat2d_in_function.c

6.5 Summary

In this chapter, we learnt that

1. A 2D matrix `A[m][n]` is an array of pointers `A[i]` (with $i = 0, \ldots, m-1$), each pointing to the first element of row i. A 2D matrix is in fact a double pointer and following the good practice introduced in chapter 4, it is recommended to name the matrix `ppA` rather than simply `A`.

2. `ppA[i][j]` is a syntactic sugar of *(*ppA+i*n+j).

3. As a consequence, it is absolutely necessary to specify the number `n` of columns when passing a 2D matrix to a function.

4. The only way to define a 2D matrix `ppA[m][n]` of $m \times n$ contiguous elements is

   (a) to declare an array of dimension $m \times n$,

   (b) then make `ppA[i]` points to the first element of each row of the matrix as stored in this array.

   ```
   int *pABlock;
   int **ppA;
   pABlock=(int *) malloc(m*n*sizeof(int));
   ppA=(int **) malloc(m*sizeof(int *));
   for(i=0;i<m;i++) ppA[i]=&pABlock[i*ncol];
   ```

5. In order to modify a 2D matrix in a function, a triple pointer must be used.

6.6 Exercices

1. Write a program that performs the following tasks
   (a) ask for two positive integers `m` and `n`
   (b) create on the `heap` a matrix of integers `A[m][n]`
   (c) read the elements of `A` from the screen
   (d) read the elements of `A` from a file
   (e) ask for a positive integer `p`
   (f) set to zero the first `p` lines of `A[m][n]`.
2. Write a program that performs the following tasks
   (a) ask for two positive integers `nrow` and `ncol`
   (b) create on the `heap` a matrix of integers `t[nrow][ncol]`
   (c) initialize the elements of `t[i][j]=cos(i`π`/2)+sin(j`π`/2)+i+j`
   (d) print `t`
   (e) ask for two positive integers `nrowB` and `ncolB`
   (f) create on the `heap` a matrix of integers `r[nrowB][ncolB]`
   (g) initialize the elements of `r[i][j]=cos(i`π`/2)+sin(j`π`/2)+i+j`
   (h) print `r`
   (i) create on the `heap` a matrix of integers `s[nrow][ncolB]`
   (j) store in `s` the product of matrix `t` and matrix `r`
   (k) print `s`
   (l) free the memory before termination.
3. Modify the code of exercice 1 so that I/O are in the main code and all the other operations are in separate functions.

array

Chapter 7

Characters and strings

7.1 Type `char`

Character variables are defined with the type `char` and handled with functions defined in the `ctype.h` header.

The format to input and output a character is %c.

There is also a function to read a character from the stdin, `getchar()`, and a function to print a character to the stdout, `putchar(...)`.

```c
#include <stdio.h>
#include <ctype.h>
int main()
{
  char C;
  printf("Please enter a letter, a digit, a sign or a space: ");
  C=getchar();   // equivalent to: scanf("%c",&C);
  printf("You wrote ");
  putchar(C);    // equivalent to: printf("%c",C);
  printf("\n");
  if (islower(C)) printf("%c is in lower case\n",C);
  if (isupper(C)) printf("%c is in upper case\n",C);
  if (isalpha(C)) printf("%c is alphabetic\n",C);
  if (isdigit(C)) printf("%c is a digit\n",C);
  if (isalnum(C)) printf("%c is alphanumeric\n",C);
  if (ispunct(C)) printf("%c is not alphanumeric\n",C);
  if (isspace(C)) printf("%c is a space\n",C);
  if (isalpha(C) && isupper(C)) printf("Converting %c into lower case: %c\n",C,tolower(C));
  if (isalpha(C) && islower(C)) printf("Converting %c into upper case: %c\n",C,toupper(C));
  return 0;
}
```

CharString/1_char.c

Characters are encoded in 1 byte, there are thus $2^8 = 256$ different ASCII characters, whose chart can be obtained with the command `man ascii`. For instance, the digit '0' corresponds to the 48^{th} character, 'A' to the 65^{th} character, 'a' to the 97^{th} character.

The ASCII numeric values should never be directly used in place of the characters but they are useful when comparing their positions in the chart:

1. 'a'+1 corresponds to the character after 'a', that is 'b'
2. 'a' < 'b' is true as 'a' is before 'b'
3. One can loop over the alphabet using: `for(C='a';C<'z';C++) ...`

```c
#include <stdio.h>
#include <ctype.h>
int main()
{
  char C;
  for (C='a';C<='z';C++) putchar (C);
  printf("\n");
  for (C='A';C<='Z';C++) putchar (C);
  printf("\n");
  return 0;
}
```

CharString/2_char_op.c

Useful special characters are:

\t	tabulation
\r	return to the same line
\n	new line
\\	\
\'	'
(char)7	BIP

7.2 String of characters

A strings is an array of characters followed by the null character '\0'. The header defining the functions to handle strings is `string.h`.

When declaring and initializing a string, the length of the array can be omitted. It can also be declared as a pointer of type `char`: an array will be allocated and initialized with the characters defining the string and the pointer will be initialised with the address of the first character.

```c
#include <stdio.h>
#include <string.h>
// How to define and handle characters strings
int main()
{
  char prenom[15];
  char nom[15];
  char prenom_nom[40];
  char *space=" ";
  printf(" What is your first name ?\n");
  scanf("%s",prenom);
  printf(" What is your family name ?\n");
  scanf("%s",nom);
  printf(" Hello %s %s \n", prenom, nom);
  strcpy(prenom_nom,prenom);
  strcat(prenom_nom,space);
  strcat(prenom_nom,nom);
  printf(" Hello %s !\n",prenom_nom);
  return 0;
}
```

CharString/3_string.c

Since strings are arrays, they can be handled using pointers. The following program defines strings using either an array (`txt`) or a pointer (`p`). The string `txt` is printed, first using the %s format, then using a pointer and the `putchar` function. It is also printed, omitting succesfully the first, second, ... , penultimate character. Finally, the string `p` is printed, omitting the last character.

```c
#include <stdio.h>
#define N 10
// Character string, array and pointer
int main()
{
  int i;
  char txt[]="Hello!";
  char *p = "Hello!";
  char *q;
  printf ("txt = %s\n",txt);
  for (q=&txt[0]; *q != '\0';q++) printf("%s\n",q);
  for (q=p; *q != '!';q++) putchar(*q); // equivalent to: printf("%c",*q);
  printf("\n");
  return 0;
}
```

CharString/4_text.c

```c
// Different methods to compare strings
#include <stdio.h>
#include <string.h>
#define M 20
int main()
{
   char txt1[M], txt2[M];
   printf("Enter a string of not more than %d characters\n",M);
   scanf("%s",txt1);
   printf("Enter a string of not more than %d characters\n",M);
   scanf("%s",txt2);
   printf("Using strcmp:\n");
   if( strcmp(txt1,txt2) == 0 )
      printf("Entered strings are equal.\n");
   else {
     printf("Entered strings are not equal.\n");
     printf("first  string = %s\n",txt1);
     printf("second string = %s\n",txt2);
   }
   printf("Without strcmp:\n");
   int i = 0;
   while ( txt1[i] == txt2[i] ){
     if ( txt1[i] == '\0' || txt2[i] == '\0' ) break;
     i++;
   }
   if ( txt1[i] == '\0' && txt2[i] == '\0' )
     printf("Entered strings are equal.\n");
   else {
     printf("Entered strings are not equal.\n");
     printf("first  string = %s\n",txt1);
     printf("second string = %s\n",txt2);
   }
   printf("Using pointers:\n");
   char *p1, *p2;
   p1 = txt1;
   p2 = txt2;
   while (*p1 == *p2){
     if (*p1 == '\0' || *p2 == '\0') break;
     p1++;
     p2++;
   }
  if ( *p1 == '\0' && *p2 == '\0' )
    printf("Entered strings are equal.\n");
  else {
     printf("Entered strings are not equal.\n");
     printf("first  string = %s\n",txt1);
     printf("second string = %s\n",txt2);
  }
   return 0;
}
```

CharString/5_strcmp.c

7.3 Exercice

Write a code to sort alphabetically a list of names.

Chapter 8

Structure

8.1 Introduction

Computers are powerful when they perform the same tasks on a large set of data stored in contiguous zones of memory. A typical example is matrix algebra, where highly optimized libraries of common operations such as addition, multiplication, inversion, etc ... are called to manipulate large matrices. These however are submitted to strong constraints such as

1. all elements must be of the same type, for instance double
2. the dimensions of the matrix are fixed
3. the order of the elements is fixed
4. indices always start at 0.

In C language, it is possible to group in memory several variables of different types (`int`, `double`, `char`, ...) under one name, and to access them using one pointer. Such an object is called `struct`.

8.2 Syntax

A structure is declared using the reserved name `struct`.

The elements of a structure are accessed by using the name of the structure followed by "." followed by the name of the element.

The number of bytes occupied in memory by the structure is obtained using the function `sizeof`.

Method 1

```
struct name_of_structure
{type name_variable_1;
 type name_variable_2;
 type name_variable_3;
 ...
} var1,var2;
```

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
  struct planet
  {
    char *name;
    int order,n_moon;
    double a,mass,radius;
  } Earth, Mars;

  Earth.name="Earth";
  Earth.order=3;
  Earth.n_moon=1;
  Earth.a=1.49598e11;
  Earth.mass=5.972e24;
  Earth.radius=6.371e6 ;
  printf("%s %12.5ekg a=%12.5em R=%12.5em order:%2d %2d moon(s) \n",Earth.name,
         Earth.a,Earth.mass,Earth.radius,Earth.order,Earth.n_moon);
  printf("size=%ld\n",sizeof(Earth));

  Mars.name="Mars";
  Mars.order=4;
  Mars.n_moon=2;
  printf("%s order:%2d %1d moons \n",Mars.name,Mars.order,Mars.n_moon);

  return 0;
}
```

Structure/struct1.c

This first method is useful if few variables need to be created.

Method 2

```
struct name_of_structure
{type name_variable_1;
 type name_variable_2;
 type name_variable_3;
 ...
};
struct name_of_structure var1, var2;
```

With this second method, the declaration of the type of structure and the declaration of the variables are dissociated so that new variables can be created by the command

```
struct name_of_structure name_of_variable
```

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
  struct planet
  {
    char *name;
    int order,n_moon;
    double a,mass,radius;
  };

  struct planet Earth;

  Earth.name="Earth";
  Earth.order=3;
  Earth.n_moon=1;
  Earth.a=1.49598e11;
  Earth.mass=5.972e24;
  Earth.radius=6.371e6 ;
  printf("%s %12.5ekg a=%12.5em R=%12.5em order:%1d %1d moon \n",Earth.name,
         Earth.a,Earth.mass,Earth.radius,Earth.order,Earth.n_moon);

  struct planet Mars;
  Mars.name="Mars";
  Mars.order=4;
  Mars.n_moon=2;
  printf("%s order:%2d %1d moons \n",Mars.name,Mars.order,Mars.n_moon);

  return 0;
}
```

Structure/struct2.c

Method 3

```
typedef struct
{type name_variable_1;
 type name_variable_2;
 type name_variable_3;
 ...
} name_of_structure;
name_of_structure var1, var2;
```

`typedef` is a reserved keyword in C language that creates an alias for another data type. The advantage is that it is no longer necessary to repeat `struct` before the structure name when declaring a variable as in Method 2, variables can be declared using the same syntax as native types

```
name_of_structure name_of_variable
```

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
  typedef struct
  {
    char *name;
    int order,n_moon;
    double a,mass,radius;
  } planet;

  planet Earth;

  Earth.name="Earth";
  Earth.order=3;
  Earth.n_moon=1;
  Earth.a=1.49598e11;
  Earth.mass=5.972e24;
  Earth.radius=6.371e6 ;
  printf("%s %12.5ekg a=%12.5em R=%12.5em order:%1d %1d moon \n",Earth.name,
         Earth.a,Earth.mass,Earth.radius,Earth.order,Earth.n_moon);

  planet Mars;
  Mars.name="Mars";
  Mars.order=4;
  Mars.n_moon=2;
  printf("%s order:%2d %1d moons \n",Mars.name,Mars.order,Mars.n_moon);

  return 0;
}
```

Structure/struct3.c

8.3 Pointer to a structure

A structure can be used as any other type of variable. It is thus possible to define a pointer to a structure.

To access an `element` of a structure addressed by a `pointer`, it is more convenient to use `->` rather than `(*pointer).element`.

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
  typedef struct
  {
    char *name;
    int order,n_moon;
    double a,mass,radius;
  } planet;

  planet Earth;

  Earth.name="Earth";
  Earth.order=3;
  Earth.n_moon=1;
  Earth.a=1.49598e11;
  Earth.mass=5.972e24;
  Earth.radius=6.371e6 ;

  planet *p;
  p=&Earth;
  printf("%s %12.5ekg a=%12.5em R=%12.5em order:%1d %1d moon \n",p->name,
         p->a,p->mass,p->radius,p->order,p->n_moon);

  //  printf("%s %12.5ekg a=%12.5em R=%12.5em order:%1d %1d moon\n",
  //         (*p).name,(*p).a,(*p).mass,(*p).radius,(*p).order,(*p).n_moon);

  planet Mars;
  Mars.name="Mars";
  Mars.order=4;
  Mars.n_moon=2;
  p=&Mars;
  printf("%s order:%2d %1d moons \n",p->name,p->order,p->n_moon);

  return 0;
}
```

Structure/pointer_to_struct.c

8.4 Dynamical allocation of structures

The function `malloc` or `calloc` can be used to dynamically allocate structures. The size of the structure is obtained with the function `sizeof`. The structure is deallocated with the `free` command.

```c
#include <stdio.h>
#include <stdlib.h>
int main(){
  typedef struct
  {
    int year,month;
  } date;

  int i,nb_elem;
  date *p;

  printf("enter number of elements of structures date : ");
  scanf("%d",&nb_elem);

  p=malloc(nb_elem*sizeof(date));
  //  p=calloc(nb_elem,sizeof(date));

  p[0].year=2010; p[0].month=1;
  printf("0 %d %d\n",p[0].year,p[0].month);
  for (i=1;i<nb_elem;i++){
    p[i].year=p[i-1].year+1;p[i].month=(p[0].month) + i%12;
    printf("%d %d %d\n",i,p[i].year,p[i].month);
  }

  free(p);

  return 0;
}
```

Structure/struct_alloc.c

8.5 Linked list

A linked list is a series of data in the form of a structure, each of them called a node and containing a pointer to the following node in the list.

```
typedef struct catalog
{
   char *title, *authors;
   int library_number, year;
   struct catalog *next;
} book;
```

It is extremely useful to define a list whose number and order of elements can be easily modified without actually displacing the data in memory. The caveat is that the data are no longer contiguous in memory.

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct planet_list
  {
    char name[10];
    int order,n_moon;
    struct planet_list *next;
  } planet;

int main()
{
  FILE *fpi;
  char file[] = "planet.data";
  fpi = fopen(file,"r");
  if ( fpi == NULL ) {
      printf("erreur ouverture fichier %s !\n",file);
      exit(1);
    }
  int planet_size=sizeof(planet);
  printf("planet_size=%d\n",planet_size);

  planet *newplanet,*planet1,*planet_prev;
  planet1 = malloc(planet_size);
  planet_prev = malloc(planet_size);
  fscanf(fpi,"%s %d %d",&planet1->name,&planet1->order,&planet1->n_moon);
  planet_prev = planet1;
  while ( !feof(fpi) )
    {
      newplanet = malloc(planet_size);
      fscanf(fpi,"%s %d %d",&newplanet->name,&newplanet->order,&newplanet->n_moon);
      planet_prev->next=newplanet;
      planet_prev = newplanet;
    }
  newplanet->next=NULL;

  planet *p = planet1;
  while (p->next != NULL) {
    printf("%u %s %d %d\n",p,p->name,p->order,p->n_moon);
    p=p->next;
  }
  return 0;
}
```

Structure/linkedlist.c

To run this code, use the data file `planet.data` provided:

```
Earth 3 1
Jupiter 5 67
Mars 4 2
Mercury 1 0
Neptune 8 14
Saturn 6 62
Uranus 7 27
Venus 2 0
```

Structure/planet.data

8.6 Typedef

To complete this chapter, another example of the use of `typedef` is given below. It helps improve the clarity of codes by defining aliases for variables with long names.

```c
#include <stdio.h>
#include <math.h>
int main() {
  int i;
  typedef double real;
  real a=M_PI,b=cos(a);
  printf ("a = %lf cos(a) = %lf\n",a,b);
  real *p; p = &a; b = *p;
  printf ("a = %lf cos(a) = %lf\n",a,b);
  typedef real tab[5];
  tab T,r={0.,1.,2.,3.,4.};
  p = r;
  for (i=0;i<5;i++) printf ("r[%d]=%lf\n",i,p[i]);
  for (i=0;i<5;i++) {
    T[i] = sqrt(r[i]);
    printf ("T[%d]=%lf\n",i,T[i]);}
  return 0;
}
```

Structure/typedef.c

8.7 Exercices

1. Matrices

   Using the following `matrix.h` file

```c
// Structure of a 2D-matrix as an array for efficiency
typedef struct
{
  int nrow;
  int ncol;
  double *mat;
} Matrix;
void mat_create(Matrix *, int, int);
void mat_init(Matrix *);
void mat_print(Matrix *);
void matsum(Matrix *,Matrix *,Matrix *);
```

Structure/matrix.h

   write a program to perform the following tasks,

   (a) Define a `struc` of matrices with `nrow` rows and `ncol` columns, and functions to allocate, initialize, print and sum two matrices. Test the functions by calling them from a `main` function that also reads the dimensions of the matrices and prints the results. It is important to note that in this exercice, the matrix is defined as an array, that is the element of the 2D-matrix `A[i][j]` corresponds to the element of the array `A[i*ncol+j]`.

   Using the following `M_matrix.h` file,

```c
// Structure of a 2D-matrix with contiguous elements

typedef struct
{
  int nrow;
  int ncol;
  double *mat_alloc;
  double **mat;
} Matrix;

void mat_create(Matrix *, int, int);
void mat_free(Matrix);
void mat_init(Matrix *);
void mat_print(Matrix);
Matrix matmul(Matrix,Matrix);
```

Structure/M_matrix.h

   write a programme that performs the following tasks:

   (a) Define a structure `Matrix` with members

   - `nrow` = numbers of rows
   - `ncol` = number of columns
   - matrix of **contiguous** `nrow` × `ncol` doubles

   (b) Read the dimensions of matrix `A[nrow][ncol]`

(c) Dynamically allocate `A`

(d) Initialize the matrix as: $A_{ij} = \cos(i * \text{M\_PI}/2) + \sin(j * \text{M\_PI}/2) + i + j$

(e) Do the same for matrix `B[nrowB][ncolB]`

(f) Multiply A by B

(g) Print the product.

2. Pascal's triangle of binomial coefficients Write a program to perform the following tasks:

(a) Ask the user to enter an integer $n > 0$

(b) Define a triangular table to store binomial coefficients

$$\left(\begin{array}{c} i \\ j \end{array} \right) = \frac{i!}{j!(i-j)!} \qquad \text{with } 0 \leq j \leq i \leq n$$

(c) Display the binomial coefficients in a clear form

(d) Ask the user to enter an integer $0 \leq k < n$ and verify the identity

$$\sum_{j=0}^{n-1} \left(\begin{array}{c} j \\ k \end{array} \right) = \left(\begin{array}{c} n \\ k+1 \end{array} \right)$$

(e) Ask the user to enter two integers $k, m \mid 0 \leq k, m \leq n$ and verify the Chu-Vandermonde identity

$$\sum_{j=0}^{k} \left(\begin{array}{c} m \\ j \end{array} \right) \left(\begin{array}{c} n-m \\ k-j \end{array} \right) = \left(\begin{array}{c} n \\ k \end{array} \right)$$

(f) Verify the identity with the Fibonacci number

$$\sum_{k=0}^{[n/2]} \left(\begin{array}{c} n-k \\ k \end{array} \right) = F(n+1)$$

(g) Transform into a function the part of the code that initializes and displays the triangular table of binomial coefficients

(h) Transform into a function the part of the code that verifies the three identities

(i) Test the two functions by calling them from a `main` program which asks for n, dynamically allocates and deallocate the table of binomial coefficients.

3. Complex numbers

Write a code that performs the following tasks:

(a) Define complex numbers in cartesian form $z = x + \mathrm{i}y$, with addition, multiplication, inversion, complexe conjugate, multiplication by a real, real power and printing.

(b) apply the functions to solve the algebraic second order equation $ax^2 + bx + c = 0$ with a, b and c real.

(c) Verify the solutions by substituting them back in the equation.

Hint: use the following header file

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
typedef struct
{
  double x;
  double y;
} cmplx;
extern cmplx add(cmplx, cmplx);
extern cmplx mult(cmplx, cmplx);
extern cmplx inv(cmplx);
extern cmplx conju(cmplx);
extern cmplx multreal(cmplx, double);
extern cmplx zpow(cmplx, double);
extern cmplx *eq2(double,double,double);
extern void verif(double, double, double, cmplx *);
extern void printz(cmplx);
```

Structure/cmplx.h

4. Modify `linkedlist.c` so that the planets are ordered by increasing value of their distance to the sun instead of alphabetically. The ordering must be done on the fly, *i.e.* by adapting the value of the `next` pointers. Print the address of the structures just after reading and after sorting to demonstrate that they are not changed.

Chapter 9

Bitwise operators

9.1 Introduction

As suggested by their names, bitwise operators works on the bits of integers' binary representation. They are fast, primitive actions directly performed by processors. They are very useful to perform efficiently certain operations such as multiplying an integer by 2, testing if an integer is a power of 2, printing the binary representation of an integer, etc.

9.2 AND operator &

Let's consider two unsigned integers, 9 and 12. The result of the bitwise operation `9 & 12` can be found by writing the binary representation of 9 and 12 and the result of the & operation between each pair of corresponding bits representing them:

Binary representation of 9	0	1	0	0	1
	&	&	&	&	&
Binary representation of 12	0	1	1	0	0
	0	1	0	0	0

The result is the binary representation of 8, hence: 9&12=8.

The & bitwise operator must not be confused with the boolean operator &&.

9.3 OR operator |

The result of the bitwise operation `9 | 12` can be found in a similar way.

Binary representation of 9	0	1	0	0	1
	|	|	|	|	|
Binary representation of 12	0	1	1	0	0
	0	1	1	0	1

The result is the binary representation of 13, hence: 9|12=13.

The | bitwise operator must not be confused with the boolean operator ||.

9.4 XOR operator ∧

The result of the bitwise operation `9 ∧ 12` can be found by applying the exclusive OR rule between each pair of corresponding bits.

Binary representation of 9	0	1	0	0	1
	∧	∧	∧	∧	∧
Binary representation of 12	0	1	1	0	0
	0	0	1	0	1

The result is the binary representation of 5, hence: 9∧12=5.

```c
#include <stdio.h>
// Bitwise boolean operators & | ^
int main()
{
  unsigned int a=9,b=12,i,j;
  for (i=0;i<2;i++)
    for (j=0;j<2;j++) printf("%u & %u = %u\n",i,j,i&j);
  printf("\n%u & %u = %u\n\n",a,b,a&b);

  for (i=0;i<2;i++)
    for (j=0;j<2;j++) printf("%u | %u = %u\n",i,j,i|j);
  printf("\n%u | %u = %u\n\n",a,b,a|b);

  for (i=0;i<2;i++)
    for (j=0;j<2;j++) printf("%u ^ %u = %u\n",i,j,i^j);
  printf("\n%u ^ %u = %u\n\n",a,b,a^b);

  return 0;
}
```

Bitwise/bitwise_boolean.c

9.5 Bitwise shift operators

The leftshift $<< n$ operator outputs the integer whose binary representation corresponds to the binary representation of the operand shifted by n places.

For example, the result of $5 << 2$ can be found by considering the binary representation of 5, 101, and by shifting all the bits by two places to the left and filling in the empty leftmost positions with 0, 10100, which is the binary representation of 20. As expected, this is 5 multiplied by 4.

The corresponding rightshift operator is $>> n$. The n leftmost positions are lost. This operator divides the operand by powers of 2 in a faster way than the corresponding arithmetic operation.

```c
#include <stdio.h>
// Bitwise shift operators << and >>
int main()
{
  unsigned int i,j,s;
  i = 1;
  for (j=1;j<10;j++) {
    s = i << j;
    printf("%u << %u = %u \n",i,j,s);
  }
  i = 5;
  s = i << 2;
  printf("\n%u << 2 = %u \n\n",i,s);
  s = 1 << 9;
  for (i=1;i<11;i++) {
    j = s >> i;
    printf("%u >> %u = %u\n",s,i,s>>i);
  }
  i = 15;
  s = i >> 1;
  printf("\n%u >> 1 = %u \n\n",i,s);
  return 0;
}
```

Bitwise/bitwise_shift.c

It must be noted that the bitwise shift operators do not modify the operand.

9.6 Bitwise $\sim$ operator

The complement operator $\sim$ outputs the integer whose binary representation corresponds to the binary representation of the operand with 1 flipped into 0 and 0 flipped into 1.

Foe example, ~ 0 corresponds to the integer whose binary representation is the series of 1 only, that is the largest representable integer. The sum $0+(\sim 0)+1$ leads to an overflow, the leftmost 1 is lost and the result is 0. It is easy to see that a similar result is obtained for any integer other than 0.

It must be noted that the $\sim$ operator does not modify the operand.

```c
#include <stdio.h>
// Bitwise complement operator ~
int main()
{
  unsigned int i;
  for (i=0;i<2;i++) printf("~%u = %u\n",i,~i);
  i = 9;
  printf("\n%u + (~%u) + 1 = %u\n\n",i,i,i+~i+1);
  return 0;
}
```

Bitwise/bitwise_complement.c

9.7 Exercices

1. Write a code to determine if an integer is even or odd using only a bitwise operator.
2. Write a code to swap two integers using only a bitwise operator.
3. Write the function
   - `void bin_rep(unsigned int x, unsigned int nbit)` to print the `n` bit representation of an integer `x`
   - `unsigned int get_bit(unsigned int x, unsigned int n)` to recover the value of the $\texttt{n}^{\text{th}}$ bit of `x`
   - `void set_bit(unsigned int *x, unsigned int n)` to set to 1 the $\texttt{n}^{\text{th}}$ bit of `x`,
   - `void unset_bit(unsigned int *x, unsigned int n)` to set to 0 the $\texttt{n}^{\text{th}}$ bit of `x`,

   the first bit being the rightmost as in the usual positional notation where the bit of strongest weight is to the left.
4. Write a programme `bitop.c` to test these functions on a series of `M` pseudo-random integers. The number `M` must be asked to the user. The `set_bit` et `unset_bit` fonctions must be called from within the loop over the `M` integers.